

A1 Brick 2

From a Nozzle Generator to a Nozzle Optimizer
The MOC-adjoint engine on the differentiable march [F2/A1]

DIMA – Sapienza University of Rome

August 9, 2026

Abstract

This document records the construction of a certified nozzle *optimizer* for rotating detonation engines. Its starting point is a flow solver, delivered earlier, that computes the supersonic flow through a given nozzle and — because it is written in a differentiable language — also returns the exact derivative of any computed result with respect to any input, at roughly the cost of one extra solve. This report is the climb from that solver to an optimizer: a thrust objective computed by two independent routes, its gradient, a classical optimality condition turned into a working instrument, a first blind optimization checked against a closed-form answer, the structural inversion that makes the wall shape an input rather than an output, an independent Fortran code brought in as referee, a term-by-term confrontation of Rao’s 1958 hand-derived optimality conditions with the automatic adjoint on a real thrust-optimized contour, the cycle layer that makes the engine an RDE tool, the plug (aerospike) march with its free exhaust boundary, the question of where to cut a truncated plug when the chamber pressure swings, a full-field confrontation with the independent code on its own designed spike, and the extension to swirling exhaust.

Every step carries its own certified test suite. Every tolerance is derived — typically by the computation measuring its own discretization error — rather than chosen; every suite contains deliberate corruptions that it must catch. All numbers quoted are results of record from actual runs.

The first three chapters assume no background whatsoever: what a nozzle is and why it has an optimum, how the solver computes, and how this program decides that something is true. The engine chapters follow, then a status ledger; every problem met on the way — the bugs the tests caught, with diagnosis and fix — is collected in the appendix.

Contents

1	Introduction	5
1.1	What this document is	5
1.2	Who this document is written for	5
1.3	The two ground rules	5
1.4	The independent referee	6
1.5	The road through this document	6
1.6	Environment and provenance	7
2	What a nozzle does, and what “optimal” means	8
2.1	The machine	8
2.2	The numbers that describe a design	8
2.3	Thrust, and why there is an optimum	9
2.4	Three families of nozzle, and why the third is hard	10
2.5	The rotating detonation engine, and the cycle	10
2.6	What “optimizing” means here, and what an adjoint buys	11
3	How the solver computes	13
3.1	The one physical fact the method exploits	13
3.2	The unit process: one cell	13
3.3	Columns, boundaries, and feet	14
3.4	Sharp corners: the one thing a cell cannot do	14
3.5	Real gas, and where the derivatives come from	15
4	How this program decides that something is true	16
4.1	Rule one: no tolerance is chosen by hand	16
4.2	Rule two: every suite must be able to say no	16
4.3	How results are reported	17
4.4	Symbols used throughout	18
5	The score and the compass	20
5.1	The thrust functional, by two routes	20
5.1.1	The question	20
5.1.2	How it works, in full	20
5.1.3	What was built	21
5.1.4	Results of record	22
5.2	The optimality meter	22
5.2.1	The question	22
5.2.2	How it works, in full	22
5.2.3	What was built	23
5.2.4	Results of record	23

6	The first optimization, against a known answer	24
6.1	The question	24
6.2	How it works, in full	24
6.3	What was built	25
6.4	Results of record	25
7	The wall becomes an input	26
7.1	The question	26
7.2	How it works, in full	27
7.3	What was built	27
7.4	Results of record	27
8	The referee, and Rao's multipliers vs the adjoint	29
8.1	The referee comes back online	29
8.1.1	What you need to know first	29
8.1.2	The question	29
8.1.3	How it works, in full	29
8.1.4	Results of record	30
8.2	Rao's multipliers vs our adjoint, on a real optimal contour	30
8.2.1	The question	30
8.2.2	How it works, in full	31
8.2.3	Results of record	31
9	The cycle layer: the engine becomes an RDE tool	34
9.1	Cycle-averaged optimization and its oracles	34
9.1.1	What you need to know first	34
9.1.2	The question	34
9.1.3	How it works, in full	34
9.1.4	Results of record	35
9.2	The brokered compromise, on a meter	35
9.2.1	The question	35
9.2.2	How it works, in full	36
9.2.3	Results of record	36
10	The plug march	38
10.1	The question	38
10.2	How it works, in full	38
10.2.1	The topology, and the one new piece of physics	38
10.2.2	Why the lip fan is constructed, not marched	39
10.2.3	The march: columns, growth at the edge, consumption at the wall	40
10.2.4	The oracle	41
10.2.5	The axisymmetric stage	41
10.3	What was built	41
10.4	Results of record	41
11	The truncated plug under the cycle	43
11.1	The question	43
11.2	How it works, in full	43
11.2.1	What the theory claims, and what it assumes	43
11.2.2	Averaging the problem is not averaging the condition	44
11.2.3	One march per phase covers every truncation	44
11.2.4	The world	44

11.3	What was built	45
11.4	Results of record	45
12	The GENO full-field plug twin	47
12.1	The question	47
12.2	How it works, in full	47
12.2.1	What you need to know first	47
12.2.2	Two independent constructions	47
12.2.3	The posing: all data GENO's own	48
12.2.4	Thresholds: declared, with provenance	48
12.3	What was built	49
12.4	Results of record: 8/8 PASS	49
12.5	The finding: a measured, attributed-open cross-code gap	50
13	Free-vortex swirl	51
13.1	The question	51
13.2	How it works, in full	51
13.2.1	Why free vortex is the one free extension	51
13.2.2	The implementation is two changes, and only two	52
13.2.3	What this is, and is not: the spiral phase pattern	52
13.2.4	The worlds	53
13.3	What was built	53
13.4	Results of record: 9/9 PASS	53
14	Comparing configurations, and the inlet angle	55
14.1	The question, and why it is mostly a bookkeeping question	55
14.2	Results of record	55
14.3	Why they are so close: the thermodynamic ceiling	56
14.4	A certification gap the comparison exposed	57
14.5	The radial-flow oracle	57
14.6	The instrument, not the flow	58
14.7	The inlet angle as a design variable	58
15	Status, what remains, and reproduction	60
15.1	Status of every suite	60
15.2	Design inputs and constraints, as of today	60
15.2.1	What a design run can fix	60
15.2.2	Which constraints can be held simultaneously	61
15.3	What remains, in order	61
15.4	Reproduction	62
A	The problems met, and how they were fixed	63
A.1	Step 1: the unclosed control surface	63
A.2	Step 6: the rejector corrected by physics	63
A.3	Step 4: the table-kink ripple	64
A.4	Step 7: three cycle-layer traps	64
A.5	Step 7 (assembly): the free-jet sign trap	64
A.6	The plug assembly: four structural lessons	65
A.6.1	Lesson 1: the degenerate edge root	65
A.6.2	Lesson 2: rows must grow with the jet	66
A.6.3	Lesson 3: foot adjacency vs step accuracy	66
A.6.4	Lesson 4: the ghost rows, and why one oracle is not enough	67

A.7	Step 10: the posing lessons of the plug cycle	67
A.7.1	The valley spike	68
A.7.2	The plug feels the ambient: the shock-free cycle bound	68
A.7.3	The theory rejecting the wrong test, twice	69

Chapter 1

Introduction

1.1 What this document is

This is the record of building a *nozzle optimizer*: a program that does not merely compute the flow through a given rocket nozzle, but searches for the nozzle shape that produces the most thrust — and can certify that what it found satisfies the mathematical conditions of an optimum.

The target application is the rotating detonation engine (RDE), in which combustion runs continuously around an annular chamber instead of sitting still. That makes the design problem harder in a specific way: the nozzle is handed not one operating condition but a periodic family of them, thousands of times a second. Everything this document builds is aimed at that difficulty.

The work is organized in two *bricks*. Brick 1, completed earlier, is the flow solver: a method-of-characteristics march written in a differentiable programming language, checked against closed-form solutions and against an independent Fortran design code. Brick 2 — this document — is the climb from a solver to an optimizer, step by step, each step carrying its own certified test suite.

1.2 Who this document is written for

No prior exposure to nozzle design, gasdynamics, the method of characteristics, optimization theory, or rotating detonation engines is assumed. The three chapters that follow supply all of it:

- Chapter 2 — what a nozzle is and does, what the design variables are, why an optimum exists at all, what an RDE is and why it creates a cycle, and what a gradient and an adjoint buy you.
- Chapter 3 — how the solver computes: the method of characteristics, cells, columns, boundaries, and why sharp corners must be handled separately.
- Chapter 4 — how this program decides that something is true, and the symbol table (Table 4.1) for the whole document.

Readers already fluent in these can start at Chapter 5. Every later chapter opens with a short plain-language statement of what it adds, and defines any new term where it first appears.

1.3 The two ground rules

Two rules apply to every test suite in this program, and they are what makes the results claims rather than demonstrations. They are stated fully in Chapter 4; in brief:

1. **No tolerance is chosen by hand.** Every pass band is *derived* — usually by running the same computation at two resolutions and letting the scheme measure its own error. A test that passes inside a hand-picked tolerance certifies nothing.
2. **Every suite must be able to say no.** Alongside the positive checks, each suite plants deliberate corruptions that it is required to catch. If the corrupted run does not fail, the suite is worthless and the step does not count as done.

A consequence worth stating: in this program the bugs the tests catch are results too. The main chapters present each step as it now works; everything that went wrong on the way — every trap and measured structural lesson, with its diagnosis and fix — is collected in Appendix A, cross-referenced from the step it belongs to.

1.4 The independent referee

One further principle shapes the work. Alongside this program there is GENO [4], an independent Fortran nozzle design code: different language, different discretization, different thermodynamics, written by other people. By standing decision it is *never modified and never differentiated*. It plays two roles: as a *generator* it produces reference contours and flowfields that our optimizer did not make, and as a *referee* its fields are required to satisfy our own equations, and ours its numbers. Two independently built programs agreeing on one physical problem is evidence that neither can produce alone.

1.5 The road through this document

Figure 1.1 maps the whole of Brick 2: each box is a chapter, in the order the work was done and certified.

The engine is assembled in this order. Chapter 5 builds the score (thrust, computed two independent ways) and the optimality certificate. Chapter 6 closes the first optimization loop and checks it against a case whose answer is known in closed form. Chapter 7 inverts the solver so that the wall shape becomes an *input* — the structural change free-form design requires — and records a finding about differentiating tabulated gas properties. Chapter 8 brings the independent referee online and confronts Rao’s 1958 hand-derived optimality conditions with the automatic-differentiation engine, term by term, on a real thrust-optimized contour.

Chapter 9 adds the cycle layer that makes the engine an RDE tool, with a theorem-level test the implementation cannot pass by accident. Chapter 10 builds the plug (aerospike) nozzle march, whose free exhaust boundary is genuinely new physics, and certifies it against a closed-form solution. Chapter 11 takes the certified plug into the cycle and measures where a truncated plug should be cut — and why that is *not* the design for the average condition. Chapter 12 runs the strongest external check the program owns, our march on the independent code’s own designed spike, and reports one honest unresolved finding. Chapter 13 extends the march to swirling exhaust and states plainly what that extension does and does not model. Chapter 15 is the ledger: the status of every suite, what a design run can fix, what remains, and the reproduction commands.

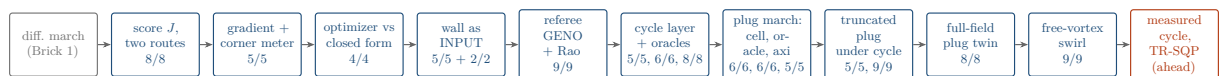


Figure 1.1: The Brick-2 pipeline. Each box is a chapter of this document and carries its own certified test suite, with its pass count shown. Blue: done and certified. Red: the road ahead. Grey: Brick 1, the prerequisite solver.

1.6 Environment and provenance

Work of record ran on host `s2` in `RDE_Design-rde-nozzle-program/` (environment `.venv-a1`: JAX 0.10.2, SciPy, Cantera 3.2.0, Matplotlib). This host’s tree is not a version-controlled checkout: files are ported to the repository for commit, with a handoff note in `validation/`. The referee code GENO was rebuilt from source on this host (Chapter 8). Every “results of record” block quotes the carrier script whose output it is, and Chapter 15 lists the one-command reproduction for each.

Chapter 2

What a nozzle does, and what “optimal” means

This chapter assumes no background at all. It explains what the machine under study is, what it is for, which numbers describe it, why there is anything to optimize, and what makes rotating detonation engines a harder design problem than ordinary rockets. A reader who already designs nozzles can skip to Chapter 3; everyone else should read it, because every later chapter uses this vocabulary without re-explaining it.

2.1 The machine

A rocket burns propellant in a chamber, producing gas at high pressure and high temperature but going nowhere in particular. The nozzle is the part that converts that stored pressure into directed, high-speed exhaust — and by Newton’s third law, the reaction to throwing that exhaust backwards is thrust.

The shape that does this is the *converging-diverging* nozzle of Figure 2.1a: the passage narrows to a minimum area, the *throat*, then opens out again. The narrowing accelerates the gas to exactly the local speed of sound at the throat; past the throat, in supersonic flow, the behaviour inverts — widening the passage accelerates the gas further. So the diverging part, which is what this document designs, keeps converting pressure into speed all the way to the exit.

Three consequences matter throughout.

The throat sets the mass flow. When the flow reaches the speed of sound at the throat, the passage is *choked*: the mass of gas per second, $\dot{m}$, is fixed by the throat area and the chamber state, and nothing downstream can change it. Holding the throat radius y_t fixed is therefore the same thing as holding the mass flow fixed — a fact this document uses as a design constraint.

The chamber state is quoted as a stagnation state. The chamber pressure P_0 and temperature T_0 (the *stagnation* values — what the gas would have if brought to rest) are the boundary data of the whole problem. Everything downstream is the same gas, expanded.

The exit is a compromise with the outside world. The gas leaves at some pressure p_e , into an atmosphere at pressure p_a . These need not be equal, and whether they are is exactly what makes design non-trivial.

2.2 The numbers that describe a design

Only a handful of quantities are needed to state a design problem, and this document keeps to them (the full symbol list is Table 4.1).

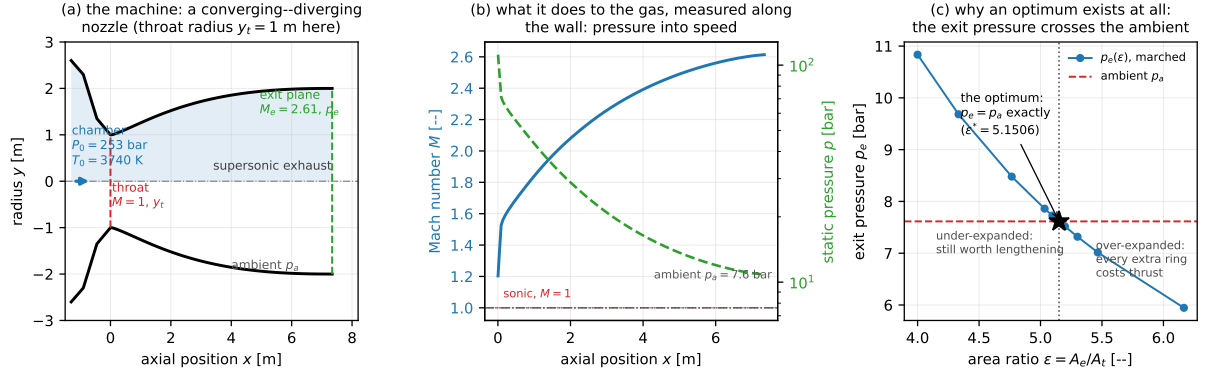


Figure 2.1: The machine, what it does, and why it has an optimum. (a) A converging–diverging nozzle: gas from the chamber (left) accelerates to sonic conditions at the throat and then supersonic in the diverging part; axes are axial position x [m] and radius y [m], with the throat radius $y_t = 1$ m in this reference case. (b) What the nozzle actually does to the gas, measured on this document’s own solver along the wall: Mach number (blue, left axis, dimensionless) rises while static pressure (green, right axis, bar, logarithmic) falls by two orders of magnitude — pressure is being traded for speed. (c) Why an optimum exists at all: the exit pressure p_e [bar] falls as the area ratio ϵ [-] grows. Above the ambient line the nozzle is still worth lengthening; below it, extra length costs thrust. The star marks where they cross — the optimum, which Chapter 6 finds *blindly* and checks against this crossing.

The *area ratio* $\epsilon = A_e/A_t$ — exit area divided by throat area — is the single most important number: it says how far the gas is expanded. A larger ϵ means a longer, wider nozzle, a faster and colder exhaust, and a lower exit pressure. The *wall contour* is the shape itself, the curve $y(x)$ from throat to exit. The *length* L is where that curve stops. And p_a , the ambient pressure, is the environment: about 1 bar at sea level, essentially zero in vacuum.

2.3 Thrust, and why there is an optimum

Thrust is a momentum balance. Written at the exit plane it is

$$J = \dot{m} u_e + (p_e - p_a) A_e, \quad (2.1)$$

the first term being the momentum carried out by the exhaust (mass flow times exit speed) and the second the net pressure force on the exit area. Chapter 5 computes this two independent ways as a self-check, but the physics of the optimum is already visible in (2.1).

Suppose the nozzle is made slightly longer, adding one thin ring of wall. Gas inside pushes on that ring with pressure p ; the atmosphere pushes back with p_a . The net gain is proportional to $p - p_a$. So while the exhaust is still at higher pressure than its surroundings — the flow is *under-expanded* — extra length pays. Once the flow has expanded past ambient — *over-expanded* — each additional ring pushes the wrong way and costs thrust. The best design sits exactly at the changeover:

$$p_e(\epsilon^*) = p_a. \quad (2.2)$$

Figure 2.1c is that statement, measured. This single idea — extend the wall while the local pressure exceeds ambient, stop when it does not — reappears throughout the document in increasingly general form: as an endpoint condition at the nozzle lip (Chapter 5), as a condition averaged over a cycle (Chapter 9), and as the rule that decides where to cut a plug nozzle short (Chapter 11).

An important special case: in vacuum, $p_a = 0$, so $p - p_a$ is positive everywhere and *longer is always better*. There is no interior optimum, only whatever length the vehicle can carry. Any

optimizer worth trusting must reproduce that behaviour, and several tests in this document check precisely that.

2.4 Three families of nozzle, and why the third is hard

The *ideal (minimum-length) nozzle* is the textbook shape: its contour is drawn so that the exhaust comes out perfectly uniform and parallel. It is easy to characterize and serves here as a reference; Brick 1 generates it.

The *thrust-optimized contour*, following Rao’s 1958 solution [1], gives up the perfectly uniform exit and asks a better question: of all walls of a *given length*, which produces the most thrust? The answer is a shorter, fuller shape, and it is what real rockets use. Rao solved it with the calculus of variations — the mathematics of optimizing over a whole curve rather than over a few numbers — and Chapter 8 confronts his hand-derived conditions with this program’s automatic machinery on exactly such a contour.

The *plug (or aerospike) nozzle* turns the geometry inside out (Figure 2.2). Instead of the gas being enclosed by a wall, the solid surface — the *spike* — sits underneath, and the outer boundary of the exhaust is nothing but the surrounding atmosphere. That free boundary is what makes the plug interesting: it is not fixed by the hardware, so the plume re-shapes itself when the ambient pressure changes. A bell is optimal at one altitude; a plug is roughly right over a range, because the exhaust always ends up at exactly p_a on its outer edge, by definition. Figure 2.2c shows this computed. Real plugs are always *truncated* — cut short and closed with a flat base — because the long thin tail carries almost no pressure but all of the weight, and where to cut is a genuine design variable (Chapter 11).

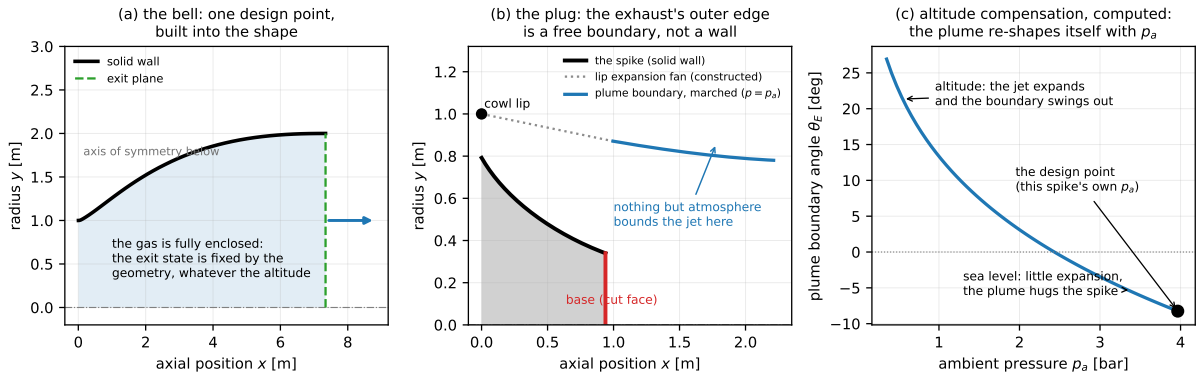


Figure 2.2: The two nozzle families this document builds, and the one physical difference between them. Axes throughout are axial position x [m] and radius y [m]. (a) The bell: the exhaust is enclosed by a solid wall, so the exit state is fixed by the geometry, whatever the altitude. (b) The plug (aerospike): the solid spike is below and the outer edge of the jet is a *free boundary* at ambient pressure. The spike shown is a real designed contour (from the independent code of Chapter 12); the plume boundary is computed by the march of Chapter 10; the red face is the base where the spike is cut short. (c) Altitude compensation, computed on that same gas: the plume boundary angle θ_E [deg] against ambient pressure p_a [bar]. At high ambient the jet barely expands and hugs the spike; as the ambient falls the boundary swings outward — the nozzle adapting itself, with no moving parts.

2.5 The rotating detonation engine, and the cycle

Everything above assumed one operating point: one chamber pressure, one ambient. A *rotating detonation engine* (RDE) breaks that assumption, and it is the reason this document has a cycle

layer.

In an RDE, combustion does not sit still and burn steadily. A detonation — a combustion wave travelling faster than sound — runs continuously *around* an annular chamber, lapping it a few thousand times a second (Figure 2.3a). Fresh propellant is injected behind the wave and consumed on the next lap. The attraction is thermodynamic: detonative combustion raises the pressure as it burns, so an RDE promises more work from the same propellant than steady burning does.

The consequence for the nozzle is severe. A fixed cross-section of the exhaust does not see a constant chamber state; it sees the pressure spike as the wave sweeps past and then decay until the next lap — over and over, thousands of times a second. What the nozzle is handed is therefore not one stagnation state but a *periodic family* of them, $(P_0(\xi), T_0(\xi))$, labelled by the position in the cycle, the *phase* $\xi \in [0, 1]$ (Figure 2.3b).

Because gas crosses the nozzle far faster than the chamber conditions change, each phase can be treated as its own steady flow, and the performance of the whole cycle is the average of the steady performances over the phases. That assumption is called *quasi-steady*; it is what makes the problem tractable with the machinery of this document, it is stated explicitly wherever it is used, and its limits are discussed honestly in Section 13.2.3.

The design difficulty is then immediate and is the whole subject of Chapters 9 and 11: each phase would want a different nozzle (Figure 2.3c), and only one piece of hardware gets built. What is the right compromise — and is it simply the nozzle for the average chamber condition? For one kind of nozzle the answer turns out to be yes, exactly; for another it is emphatically no. Measuring both is what this document does.

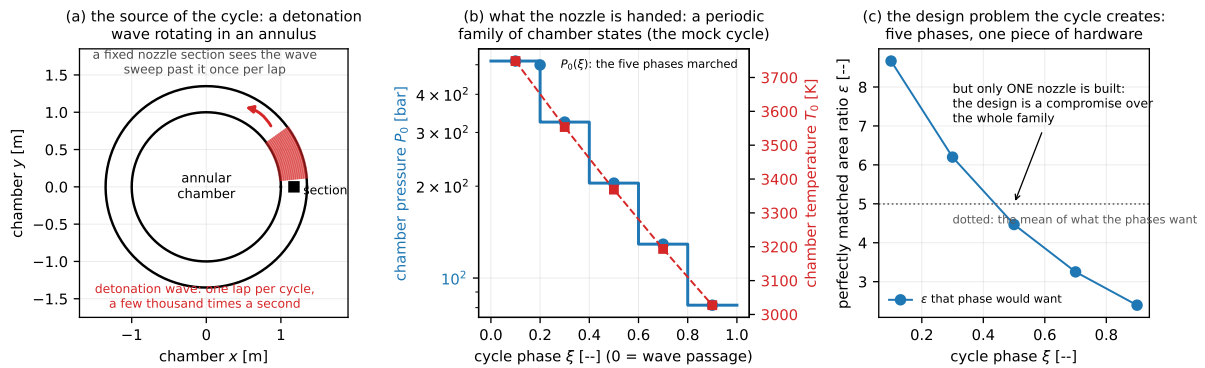


Figure 2.3: Why an RDE nozzle is a harder design problem. (a) The source of the cycle: a detonation wave runs around an annular chamber (axes are chamber cross-section coordinates x, y [m]); a fixed nozzle section sees it sweep past once per lap, a few thousand times a second. (b) What the nozzle is therefore handed: not one operating point but a periodic family. Cycle phase ξ [-] on the horizontal axis ($\xi = 0$ at wave passage), chamber pressure P_0 [bar] on the left axis (logarithmic, blue) and chamber temperature T_0 [K] on the right (red) — the five discrete phases actually marched by the carriers. (c) The resulting design problem: for each phase, the area ratio ϵ [-] that would be perfectly matched for *that* phase, computed with the program’s own gas tables. They spread by a factor of three, and only one nozzle gets built.

2.6 What “optimizing” means here, and what an adjoint buys

An *optimizer* needs three things a flow solver does not have. A *score*: one number saying how good a design is — here the thrust J , built in Chapter 5. A *compass*: the derivative of that score with respect to every design variable, the *gradient*, which says which way to move. And a *certificate*: a condition that recognizes an optimum when it is reached, so that “the search

stopped” and “the design is optimal” are different statements.

The gradient is where the technology of Brick 1 matters. If a design has only one knob, the gradient can be estimated by *finite differences* — change the knob a little, see how J changes. With a whole wall shape as the unknown, that costs one extra flow solution per degree of freedom, which is hopeless. *Automatic differentiation* (AD) avoids this. Because the solver is written in a language that records the arithmetic it performs, the chain rule can be applied backwards through the entire computation, yielding the derivative of J with respect to *all* inputs at once, for roughly the cost of one extra solve, and to machine accuracy rather than to the accuracy of a difference quotient. This backwards pass is mathematically the same object as the *discrete adjoint* of classical optimal-control theory — hence the name of this work package — but it is obtained for free rather than derived and coded by hand.

That is the engine. The rest of the document builds an optimizer around it, one certified piece at a time.

Chapter 3

How the solver computes

The rest of this document talks constantly about “cells”, “columns”, “feet”, “characteristics” and “compatibility”. This chapter defines all of them from nothing. It describes the *method of characteristics*, the numerical method the whole program rests on, and it is worth the four pages: almost every technical lesson recorded later — including every bug in the appendix — is a statement about the objects introduced here.

3.1 The one physical fact the method exploits

In a gas moving *slower* than sound, a disturbance spreads in every direction, upstream included: the whole flow field is coupled to itself, and solving it means solving everywhere at once. In a gas moving *faster* than sound, it cannot. A disturbance made at a point is swept downstream faster than it can propagate, so its influence is confined to a wedge opening downstream of that point.

That wedge has a precise half-angle, the *Mach angle* α , given by

$$\sin \alpha = \frac{1}{M}, \quad M = \frac{q}{c}, \quad (3.1)$$

where q is the flow speed, c the local speed of sound, and M their ratio, the *Mach number*. Faster flow means a narrower wedge. Its two edges, drawn through the point, are the *characteristics*: one at angle $\theta + \alpha$ to the axis and one at $\theta - \alpha$, where θ is the direction the gas is actually flowing (Figure 3.1a). By convention the first family is called C^+ and the second C^- .

Characteristics are not just geometry. Along each of them the flow must satisfy a *compatibility relation*: an ordinary differential equation tying the change in speed to the change in flow direction, with an extra term in axisymmetric flow accounting for the passage opening out as the radius grows. That is the entire physical content the solver uses. Two families of lines, one relation along each — no volume grid, no time stepping, no iteration to steady state.

3.2 The unit process: one cell

The method turns those relations into arithmetic with a single repeating move, the *cell* (or *unit process*), shown in Figure 3.1b.

Suppose two nearby points are already solved: position and velocity known at both. From the lower one, draw its C^+ characteristic; from the upper one, its C^- . In supersonic flow these two lines converge and cross at exactly one point downstream. That crossing gives two equations, one per line, for the two coordinates of the new point. The two compatibility relations, integrated along the two short segments just drawn, give two more equations for the two velocity components there. Four equations, four unknowns (x, y, u, v) .

The equations are nonlinear — the characteristic directions depend on the very state being solved for — so each cell is closed by a small Newton iteration. Two details of how they are closed matter later:

Midpoint coefficients. The coefficients of the compatibility relations are evaluated not at the segment’s start but at its *midpoint*, using the average of the two endpoint states. This is the trapezoidal rule; it makes the error per cell shrink with the square of the step rather than linearly, which is why the whole march is called second order. It is also the reason a cell must never span a region where the state changes abruptly: averaging the two ends of such a segment is then meaningless (Chapter 10 contains a measured example).

Newton certification. Once a cell’s Newton iteration converges, the program checks that the residual of the four equations really has fallen to the level of floating-point roundoff, scaled to the size of the unknowns. A cell that stalls is caught and reported rather than quietly polluting the march. Every “all cells Newton-certified” claim in this document is that check, run over every cell.

3.3 Columns, boundaries, and feet

New points grow out of solved points, so a march organizes itself into *columns*: from one solved column, the interior cells produce the next column, and the sweep repeats downstream (Figure 3.1c). A march therefore needs somewhere to begin — a *start line* (or initial-value line) across the flow, on which the complete state is supplied as data. In a rocket nozzle that line sits just downstream of the throat.

At the edges of the flow the cell changes shape, because one of its two parent characteristics is missing and a boundary condition takes its place. Three boundaries appear in this document.

A symmetry axis: the flow cannot cross it, which supplies the missing relation.

A prescribed wall: the new point’s position is already known, because it must lie on a given contour with a given slope. The cell then works *backwards*. It must find the *foot* — the point on the previous column from which the arriving characteristic departed — which in general falls between two stored points and must be located by searching along the column and interpolating. The compatibility relation is then closed along the chord from that foot to the wall point. This is the cell that lets a designer hand in an arbitrary wall (Chapter 7), and the accuracy of the foot search is a recurring theme.

A free jet boundary: the case with no wall at all, where the exhaust is bounded by the atmosphere. Here the situation is exactly reversed: the *pressure* is known (it is p_a , since nothing but the atmosphere pushes on the jet) while the *position* of the boundary is unknown and must be solved. This is the one genuinely new piece of physics the plug nozzle needs, and Chapter 10 builds and certifies it.

3.4 Sharp corners: the one thing a cell cannot do

Where a wall turns a sharp corner, the flow expands through a *centered expansion fan*: a continuous spread of characteristics all emanating from that single geometric point, each carrying a different state. At the corner itself the flow is therefore multivalued and the gradients are infinite.

No ordinary cell can step across such a fan, for both of the reasons given above: a segment spanning the fan would average states that differ by the entire turn, and near the corner the characteristic crossing that fixes the new position degenerates because all the lines meet at one point. The cure is that a centered fan does not need to be marched — it is *self-similar*, so the state on each ray is known in closed form from the ray angle alone. The corner is therefore *constructed* as data and ordinary cells resume between the rays, where the flow is smooth again. Chapter 10 shows what happens when this rule is violated, with the measured number.

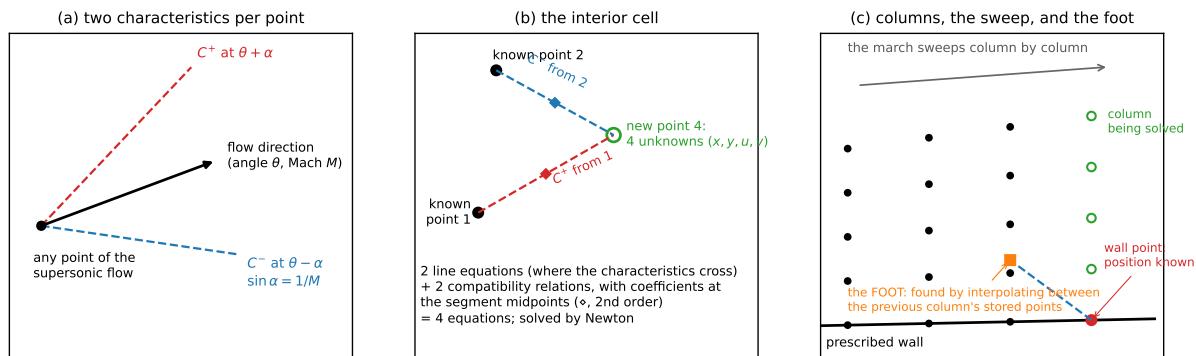


Figure 3.1: How the march computes, for a reader new to the method of characteristics. Axes throughout are axial position x [m] and radius y [m]. (a) Through every point of a supersonic flow pass two characteristics, tilted by the Mach angle α on either side of the local flow direction θ ; along each, a compatibility relation links the change in speed to the change in angle. (b) The interior cell: from two already-solved points, the crossing of a C^+ and a C^- fixes the new point's position, and the two compatibility relations — closed with coefficients taken at the segment midpoints ($\diamond$) — fix its velocity; a Newton iteration solves the resulting 4×4 system. (c) Cells build columns out of columns. At a prescribed wall the position is known instead of computed, so the cell must locate the *foot* of the arriving characteristic on the previous column by interpolation. Every structural lesson in Chapter 10 and Appendix A is a statement about these three pictures.

3.5 Real gas, and where the derivatives come from

Two implementation choices are worth stating once here.

The gas is not assumed to be calorically perfect. Thermodynamic properties come from tabulated data, and the state is recovered from the speed through the energy and entropy relations. This matters in one non-obvious way that Chapter 7 documents: tabulated properties are interpolated piecewise, so the *exact* derivative of the code carries a small ripple at the scale of the table cells — a real property of the discrete objective, not a bug, but one that makes a naive derivative test ill-posed.

And the entire march — every cell, every Newton iteration, every foot search — is written so that it can be differentiated automatically. That is what turns a solver into the engine of an optimizer, and it is the subject of everything that follows.

Chapter 4

How this program decides that something is true

A design code is only as good as the evidence that it is right. This chapter states the two rules every test suite in this document obeys, explains why they were chosen, and defines the handful of terms used to report results. It closes with the symbol table for the whole document.

4.1 Rule one: no tolerance is chosen by hand

Every check in this document compares a computed number against a band, and *no band is picked by a human*. The reason is simple: a tolerance chosen after seeing the answer certifies nothing. It always exists, and it always passes.

Instead, the band is *derived*, and in almost every case the computation derives it about itself. The same quantity is computed twice, once on a coarse discretization and once on a fine one. The difference between the two is a measurement of the error the scheme is still making. The band is that difference, multiplied by a fixed safety factor $K_{\text{RICH}} = 4$, plus a floor at the level of floating-point roundoff:

$$\text{band} = K_{\text{RICH}} | \text{coarse} - \text{fine} | + \text{roundoff floor}. \quad (4.1)$$

This is the classical idea of *Richardson extrapolation* used as an error estimate rather than as an accelerator, and the resulting number is called a *Richardson band* throughout (Figure 4.1b). Because the method is second order (Chapter 3), halving the step should cut the error by about four; when a suite reports a “coarse/fine ratio of 4.52” it is reporting that the scheme is converging at the rate it should, which is itself evidence that the band means something.

Not every quantity has a discretization to refine. Where it does not, the band is derived from something else measurable — the scatter of the reference data itself, or the physical scale of the quantity — and the source is always stated. One chapter (Chapter 12) uses *declared* thresholds rather than derived ones; it says so explicitly, gives their provenance, and explains why refinement would be the wrong ruler there. That exception is deliberate and visible, which is the point.

4.2 Rule two: every suite must be able to say no

A test that has never failed may be measuring nothing. So alongside the positive checks, each suite contains deliberate corruptions — called *rejectors* — which the suite is required to catch: a sign flipped, a term dropped, a wrong ambient pressure, a wrong characteristic foot, a geometry perturbed by a fraction of a percent. If a rejector does *not* fail, the corresponding positive check is worthless and the step does not count as done.

Rejectors do real work in this document. They are what makes a declared threshold trustworthy (they show the instrument resolves defects an order of magnitude above the gap being measured), and several of them caught genuine errors in the program before they could reach a result.

Figure 4.1a puts both rules on one picture: every check in this document, expressed as its measured deviation divided by its own band, together with every deliberate corruption. The positive checks must land left of one; the rejectors must land right of it. Nothing is close to the line by accident.

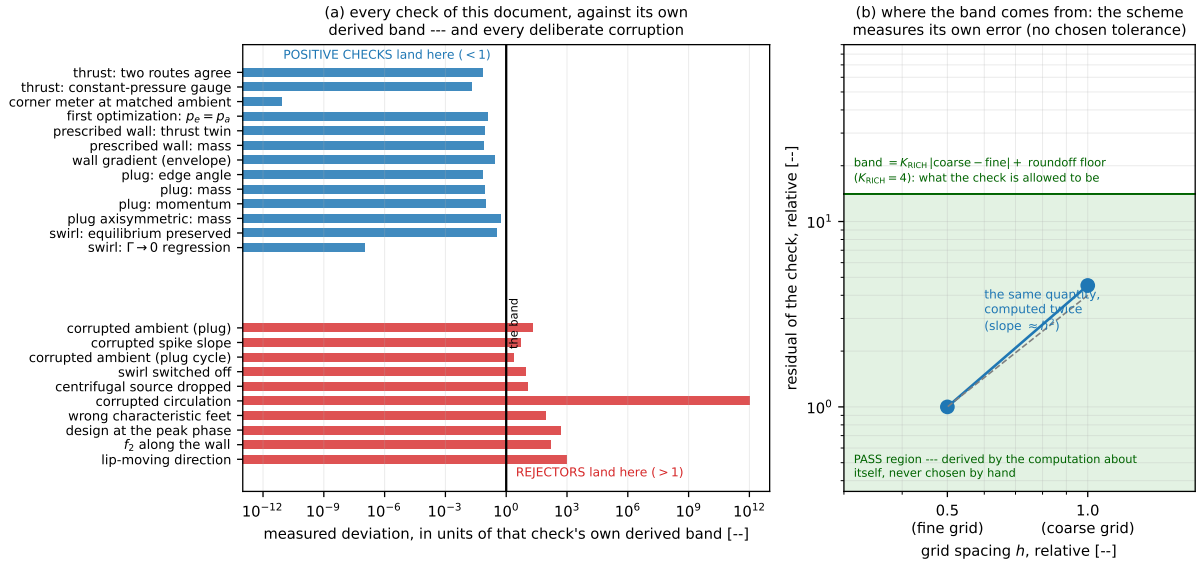


Figure 4.1: The method of this document, in one picture. (a) Every certified check (blue) and every deliberate corruption (red), plotted as the measured deviation divided by that check's *own* derived band — so the horizontal axis is dimensionless and the vertical line at one is the pass/fail threshold for all of them at once. Positive checks land far left (most by two to six orders of magnitude); rejectors land far right. (b) Where a band comes from: the same quantity is computed on a coarse and a fine grid, the difference measures the error the scheme is still making, and the band is that difference times a safety factor of four plus a roundoff floor. Axes are the grid spacing h and the residual of the check, both relative and logarithmic; the dashed line is the second-order slope the method should follow.

4.3 How results are reported

A few conventions recur.

“ m/n PASS” counts the checks in a suite, rejectors included: a suite of six positive checks and two corruptions that behaves correctly reports 8/8.

“*Newton-certified*” means every cell's converged residual was verified to sit at roundoff level, scaled to the size of its unknowns (Chapter 3). The number quoted (“worst residual 0.019”) is a fraction of the allowed tolerance, so anything below one passes.

“*Result of record*” means the number is quoted verbatim from an actual run of the named carrier script, not recomputed for the document. Chapter 15 lists the one command that reproduces each.

An *oracle* is a case whose answer is known independently of the program — a closed-form solution, an exact conservation identity, or a second code — against which the program can be judged. Where no oracle exists, a suite falls back on *internal closures*: conservation statements

(mass, momentum) that must hold whatever the answer is, plus *alive controls*, which check that a term the program claims to include actually changes the answer when removed.

Finally, and deliberately: **in this program the bugs the tests caught are results too.** The main chapters describe each step as it now works; every trap, wrong turn, and measured structural lesson is collected in Appendix A, cross-referenced from the step it belongs to. A caught bug is evidence that the instrument has teeth.

4.4 Symbols used throughout

Symbol	Units	Meaning
<i>Geometry and flow</i>		
x, y	m	axial position, radius
u, v	m/s	axial and radial velocity components
q	m/s	flow speed, $\sqrt{u^2 + v^2}$ (written W in Rao's papers)
θ	rad, deg	flow direction, $\arctan(v/u)$
p, ρ, T	Pa, kg/m ³ , K	static pressure, density, temperature
c	m/s	local speed of sound
M	–	Mach number, q/c
α	rad	Mach angle, $\sin \alpha = 1/M$
C^+, C^-	–	the two characteristic families, at $\theta + \alpha$ and $\theta - \alpha$
$\nu(M)$	rad	Prandtl–Meyer function (the turn a corner expansion produces)
δ	–	0 for planar flow, 1 for axisymmetric
<i>The machine and its operating point</i>		
y_t	m	throat radius (holding it fixes the mass flow)
r_{tu}, r_{td}	m	throat arc radii, upstream and downstream
A_t, A_e	m ²	throat and exit area
ε	–	area ratio, A_e/A_t
L	m	wall length
P_0, T_0	Pa, K	chamber (stagnation) pressure and temperature
p_a	Pa	ambient (back) pressure
p_e, u_e	Pa, m/s	exit pressure and speed
$\dot{m}$	kg/s	mass flow
J, F	N	thrust (the objective being optimized)
<i>The cycle (rotating detonation engine)</i>		
ξ	–	cycle phase, 0 at wave passage to 1 at the end of the blowdown
μ	–	the measure that weights the phases in the cycle average
PR, TR	–	pressure and temperature ratio across the cycle
<i>Plug nozzle and swirl</i>		
l	m	truncation length: where the spike is cut short
D	–	the truncation point itself
w	m/s	azimuthal (swirl) velocity
Γ	m ² /s	circulation, yw ; constant in a free vortex
<i>Instruments</i>		
R_{mom}	N	disagreement between the two independent thrust routes
R_{corner}	Pa	endpoint (corner) residual: zero at an optimal lip
f_2	m/s	Rao's first integral, $q \cos(\theta - \alpha) / \cos \alpha$, constant along the terminal characteristic of an optimal nozzle
λ_2, λ_3	–	Rao's constant Lagrange multipliers ($f_2 = -\lambda_2$)
K_{RICH}	–	Richardson safety factor, fixed at 4

Table 4.1: Symbols used throughout this document. Lengths are quoted in metres; the reference cases are posed with a throat radius $y_t = 1$ m, so numerical values of x and y are also multiples of the throat radius.

Chapter 5

The score and the compass

The solver of Brick 1 answers “what does the gas do in this nozzle?”. An optimizer needs two further things: a single number that says how good a nozzle is, and the derivative of that number with respect to every shape knob, which tells it which way to move. This chapter builds both and, more importantly, makes both checkable — the number by computing it twice along two completely different paths, the derivative by comparing it with brute-force re-runs of the solver. It then adds a second instrument that does not score a design but *certifies* it: a dial that reads zero exactly when the nozzle wall ends where an optimal wall must end.

5.1 The thrust functional, by two routes

5.1.1 The question

An optimizer needs exactly two things the flow solver did not have. First a *number* that says how good one design is — the *objective* J , here the thrust in newtons, the axial force the nozzle produces. Second its *gradient*: the list of partial derivatives of that number with respect to each design knob, i.e. how much J changes per metre of change in each knob. The gradient is the compass; it points uphill.

5.1.2 How it works, in full

Thrust is a *momentum balance*: Newton’s second law applied not to a body but to a region of gas. Draw a *control surface* — an imaginary closed surface enclosing the nozzle flow — and add up two things on it: the momentum the gas carries through the surface, and the pressure force pushing on it. The sum is the force the machine feels. Nothing physical depends on where that surface is drawn, and that freedom is exploited twice, by computing J on two surfaces that share no data:

$$J = \dot{m} u_e + (p_e - p_a) A_e \quad (\text{route 1: read the exit}), \quad (5.1)$$

$$J = F_{\text{throat}} + \int_{\text{wall}} (p - p_a) 2\pi y \, dy \quad (\text{route 2: read the wall}). \quad (5.2)$$

Route 1 closes the surface at the exit plane, where an ideal nozzle delivers flow that is uniform and parallel to the axis. There the balance is elementary: the mass flow $\dot{m}$ [kg/s] times the exit speed u_e [m/s], plus the mismatch between the exit static pressure p_e and the ambient (surrounding) pressure p_a acting on the exit area A_e .

Route 2 closes the same surface along the machine itself. Its first term is the momentum flux entering through the *throat* — the narrowest cross-section, where the gas reaches exactly the local speed of sound. A throat in that state is called *choked*: the mass flow through it is then set by the throat area and the chamber conditions alone, and nothing downstream can increase it. The carrier evaluates this flux on the march’s initial-value line, the short line across

the throat where the solver starts. The second term is the axial push of the wall: every ring of wall of radius y and radial width dy presents an axial projected area $2\pi y dy$ on which the pressure excess $(p - p_a)$ pushes forward. The two routes share no intermediate quantity: route 1 never reads a wall pressure, route 2 never reads the exit state.

Newton’s law forces the two answers to agree, so their difference R_{mom} [N] is a mechanical error meter — if the discretization is wrong anywhere, the two routes stop matching. How large is it allowed to be? Not a number chosen by taste. The march is run twice, at one resolution and at double resolution; the *change* in R_{mom} between the two runs is the scheme’s own estimate of its discretization error. That change, multiplied by the safety factor $K_{\text{RICH}} = 4$ and added to a floor for floating-point roundoff, is the allowed band — a *Richardson band*, named after the two-resolution extrapolation idea it comes from.

Two further instruments complete the step. The *gauge test*: add the same constant to the pressure everywhere and the thrust must not move, because a constant pressure integrates to zero over any closed surface. That is an identity of the *formulas*, not of the flow, so it isolates errors in the surface bookkeeping from errors in the flow — and it caught one (Appendix A.1). And the *gradient*: because the march is written in a differentiable language, *reverse-mode automatic differentiation* (AD) — the technique that differentiates the program itself, propagating derivatives backwards through every operation it performed, which is mathematically the same as solving the discrete adjoint equations — returns dJ/dP for all design parameters $P = (y_t, r_{tu}, r_{td}, \varepsilon)$ in a single backward pass, at about the cost of one extra march. Each of the four components is then checked against a *central finite difference*: re-run the march with that knob nudged up by h and down by h , and divide the difference of the two thrusts by $2h$. Halving h gives the second estimate the Richardson band is built from.

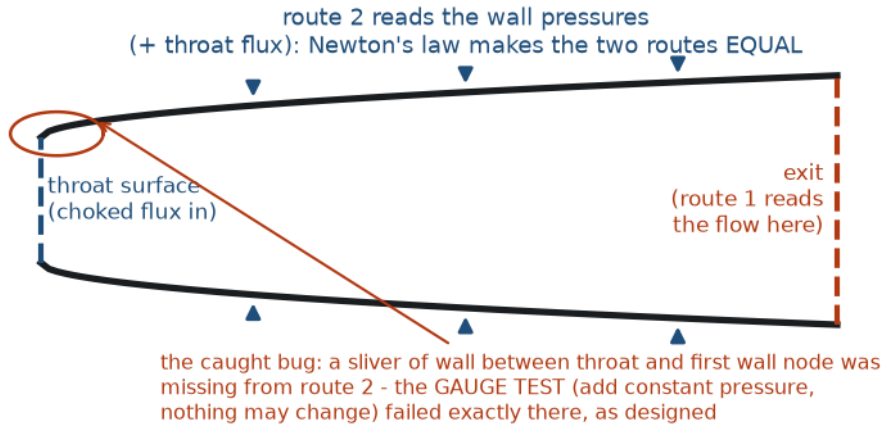


Figure 5.1: Two independent routes to one thrust, drawn in the nozzle’s own coordinates (x [m] along the axis, y [m] radius). Route 1 (right) closes the control surface at the exit plane and needs only the uniform exit state: mass flow times exit speed plus $(p_e - p_a)A_e$. Route 2 (left and along the wall) closes it on the machine: momentum flux through the choked throat plus the axial pressure push $(p - p_a) 2\pi y dy$ of each ring of wall. Look at what the two paths do *not* share: no quantity is used by both, so their difference R_{mom} [N] is a genuine error meter rather than an identity. The red circle marks the sliver of surface that was accidentally left open at first, and whose story is told in Appendix A.1.

5.1.3 What was built

`validation/a1_thrust_functional.py`: both routes, the momentum residual R_{mom} with its Richardson band, the gauge test, reverse-mode dJ/dP checked against central finite differences

knob by knob, and three rejectors — deliberately corrupted versions of the computation (dropped pressure–area term, un-closed surface, corrupted flux) that the suite must catch.

5.1.4 Results of record

8/8 PASS. $J = 1.2552079 \times 10^8$ N; $R_{\text{mom}} = +2.85 \times 10^3$ against a derived band of 4.01×10^4 (3.2×10^{-4} of J), so the two routes agree to about three parts in ten thousand of the thrust; the coarse/fine residual ratio of 4.52 is the ≈ 4 expected of a second-order scheme when the resolution is doubled, confirming that what is left is discretization error and not a mistake. Gauge delta 3.35×10^{-8} vs. allowed 1.78×10^{-6} : adding a constant pressure really does change nothing. All four gradient components fall inside their finite-difference bands (dJ/dy_t agreeing to 9 significant figures), and all rejectors fire.

The gradient signs also behave as theory predicts: $dJ/d\varepsilon > 0$ against a vacuum, i.e. with nothing pushing back a longer nozzle is always worth building; and dJ/dr_{td} is tiny, i.e. the curvature of the short arc just downstream of the throat barely matters. One real bug was caught and fixed on the way: Appendix A.1.

5.2 The optimality meter

5.2.1 The question

A score and a compass find better designs. What *certifies* that a given design already satisfies the condition an optimum must satisfy?

5.2.2 How it works, in full

Choosing the best nozzle wall is not a problem of finding the best value of a few numbers but of finding the best *curve*; the branch of mathematics for that is the *calculus of variations*, which asks what must be true of a curve if no small deformation of it can improve the objective. Rao [1] solved that problem for a nozzle and found, among the conditions, one exact scalar statement that must hold at the lip — the point where the wall stops. It is an *endpoint* (or transversality) condition:

$$R_{\text{corner}}(p_a) = (p - p_a) - \frac{1}{2}\rho W^2 \sin(2\theta) \tan \alpha. \quad (5.3)$$

Read it as the balance of the two effects of moving the lip one step further downstream. The first term is the direct gain: one more ring of wall, worth building as long as the pressure on it exceeds ambient. The second is the price: the extra wall forces the flow to keep turning, and the cost of that turning is written through the local flow angle θ (the direction of the velocity relative to the axis) and the Mach angle α (the tilt of the lines along which supersonic disturbances travel, Chapter 3). $R_{\text{corner}} = 0$ means “the wall stops in exactly the right place”: one more ring would gain as much as it costs. The meter evaluates this from the march’s own lip data, so any design the engine produces can be certified and not merely scored. Averaged over the engine cycle it becomes the lip condition that a rotating detonation engine’s nozzle must satisfy (statement T7 of [5]), which Chapter 9 puts on a meter of its own.

The companion instrument is the *first integral*

$$f_2 = \frac{W \cos(\theta - \alpha)}{\cos \alpha} = -\lambda_2, \quad (5.4)$$

where a first integral means a combination of flow quantities that the governing equations force to stay constant along some particular curve, even though each ingredient varies. Rao’s theory makes f_2 constant along the *terminal characteristic* of an optimal nozzle — the last of the two families of lines along which supersonic information travels, the one that closes the flowfield at the lip. Its constant value is $-\lambda_2$, a *Lagrange multiplier*: the number that prices a constraint

in a constrained optimization, here telling how much thrust would be gained per extra unit of mass flow allowed. Along any other curve — in particular along the wall, which is a *streamline*, a line everywhere tangent to the velocity and hence the path a gas particle follows — f_2 is *not* constant. That supplies the natural negative control: an implementation that made f_2 look constant everywhere would be broken, and must be caught.

5.2.3 What was built

`validation/a1_corner_instrument.py`: the corner meter and the f_2 instrument, tested against three known answers on the ideal-nozzle case (axial exit, matched ambient, vacuum), with the non-constancy rejector on the wall.

5.2.4 Results of record

5/5 PASS (82 s). The flow angle at the lip is machine-zero ($\theta = -10^{-16}$ rad: the exit really is axial, as the ideal nozzle assumes). With the ambient set equal to the exit pressure the meter reads 3.7×10^{-10} against an allowed 42.8 Pa — the nozzle is certified stationary at its design ambient. Against a vacuum it reads exactly $+p_e$, i.e. “lengthen me”, the statement that no finite nozzle is optimal in vacuum, now on a dial; and its sign agrees with an independently recomputed $dJ/d\varepsilon$, so meter and gradient tell the same story. The first integral f_2 at the lip equals the exit speed to 4.6×10^{-13} m/s, while along the wall it varies by 1173 m/s — the non-constancy rejector, firing as designed.

Chapter 6

The first optimization, against a known answer

The previous chapter gave the engine a score and a compass. This one closes the loop for the first time: a search that proposes designs, scores them with the march, and keeps going until it cannot improve. The point is not the answer but the audit — the problem is posed so small that its solution is known in advance on paper, so the machine can be judged against it. A loop that lands on a known answer while knowing nothing about it can be trusted on problems whose answer is not known.

6.1 The question

Does the loop close? Propose, score, search — and land where theory says the optimum is?

6.2 How it works, in full

The first optimization is deliberately one-dimensional, so that its answer exists in closed form (an exact formula, as opposed to a number obtained by iteration). The single knob is the *area ratio* $\varepsilon = A_e/A_t$ — the exit area divided by the throat area, the one number that says how much the nozzle lets the gas expand (Chapter 2). Everything else is held fixed.

The known answer comes from the exit-area lemma. Differentiate the thrust (5.1) with respect to the exit area of an ideal (uniform-exit) nozzle: enlarging the exit by dA_e adds a ring of wall on which the local exit pressure p_e pushes forward, while the ambient pressure p_a pushes back on the same ring, so

$$\frac{dJ}{dA_e} = p_e - p_a \quad \implies \quad \text{optimum exactly where } p_e(\varepsilon^*) = p_a. \quad (6.1)$$

A nozzle whose gas is still above ambient at the exit ($p_e > p_a$) is called *underexpanded*: it has not finished expanding, and is worth lengthening. Once the gas has been expanded below ambient ($p_e < p_a$) the nozzle is *overexpanded*: the surroundings now push back harder than the gas pushes forward on each new ring, and every extra ring costs thrust. The optimum is the crossover. Posing an ambient pressure therefore fixes ε^* analytically — the march only has to be inverted for the area ratio at which its exit pressure equals that ambient.

The search itself knows none of this. It is a *golden-section search*: a derivative-free method that keeps a bracket $[a, b]$ known to contain the maximum, evaluates the objective at two interior points placed by the golden ratio, discards the end that cannot hold the maximum, and repeats until the bracket is narrower than a set tolerance. Each shrink reuses one of the two previous evaluations, which is what the golden ratio buys. Its only inputs are calls to the march and

readings of J — no gradient, no knowledge of the lemma. Every march evaluation is cached to disk keyed by ε , so an interrupted run resumes at no cost, and the landscape in Figure 6.1 is drawn from the search’s own evaluations rather than from a separate sweep.

The suite then closes three independent loops around the answer. The area ratio found by the blind search must match the closed form. The exit pressure there must equal the posed ambient, within a band derived from the march’s own two-resolution error estimate plus the width of the bracket the search stopped on. And the corner meter of Section 5.2 — built two steps earlier, never consulted by the search — must independently read zero at the design that was found.

6.3 What was built

`validation/a1_driver_eps.py`: the golden-section driver, the disk cache, the three closure checks, and a rejector — a deliberately corrupted objective, with the pressure–area term $(p_e - p_a)A_e$ deleted, which must then run monotonically to the edge of the bracket instead of finding an interior optimum, because without that term nothing ever penalizes over-expansion.

6.4 Results of record

4/4 PASS (231 s, 13 march evaluations). The blind search lands at $\varepsilon = 5.1551$ against the analytic 5.1506; the exit pressure at the found optimum matches the posed ambient to 916 Pa, inside a derived band of 7958 Pa; the corner meter independently reads ≈ 0 there, so the optimality condition of Rao’s theory is satisfied at a design the search reached without ever evaluating it; and the corrupted objective runs to the bracket edge — rejected.

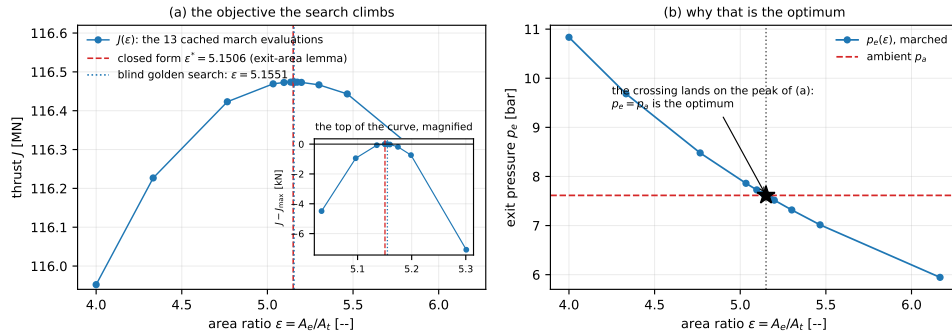


Figure 6.1: The optimization landscape and the physics behind its peak, both drawn from the driver’s own 13 cached march evaluations. (a) Thrust against shape: area ratio ε [-] on the horizontal axis, objective J [MN] on the vertical. Look at how flat the top is — the closed-form optimum (red dashed) and the blind search’s landing point (blue dotted) differ by 0.09% in ε on a curve that is flat to parts in 10^5 , so the blind search gives up less than 10^{-5} of the thrust. (b) Why the peak sits there: exit pressure p_e [bar] against the same ε [-], falling across the ambient pressure line p_a . The crossing — gas leaving above ambient to the left (underexpanded, lengthen), below ambient to the right (overexpanded, shorten) — lands on the same ε as the peak in (a). That is the exit-area lemma, measured rather than assumed.

Chapter 7

The wall becomes an input

So far the solver has drawn its own nozzle wall: the shape came out of the computation, and the only way to change it was to change the few knobs that steer the construction. An optimizer that is to explore free-form shapes needs the data flow reversed — hand it any wall, get the flow on that wall. This chapter performs that inversion using the same verified building blocks, checks the new engine against the old one on the one wall both can handle, and certifies the derivative of thrust with respect to an arbitrary wall deformation. That derivative is what every later optimization in this document steers by.

7.1 The question

Brick 1's march *draws* its wall: the contour is computed as a *streamline* — a line everywhere tangent to the local velocity, the path a gas particle follows — chosen so that exactly the throat's mass flow passes inside it. In other words the wall is an output. Free-form design needs the opposite: hand in *any* wall, get the flow on it. Can the same verified cells compute the flow on a prescribed wall?

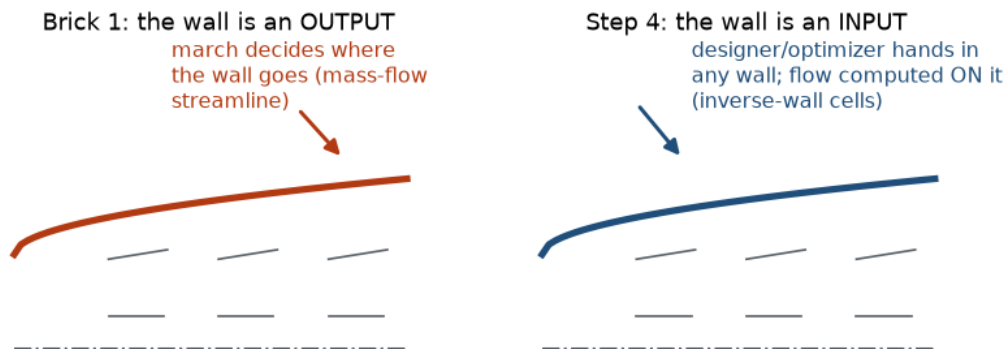


Figure 7.1: The structural inversion, drawn in nozzle coordinates (x [m] along the axis, y [m] radius). Left: the generator decides its own wall by following a mass-flow streamline, so the shape is a result. Right: the optimizer hands in an arbitrary wall and the inverse-wall cells compute the flow on it, so the shape is data. Look at the arrows: what changes is only the direction in which wall position and flow state are determined — everything upstream, and every cell interior to the flow, is unchanged. That is why the new engine inherits the old one's certification.

7.2 How it works, in full

The engine keeps everything upstream of the wall unchanged — the same throat start, the same initial expansion fan — and changes only how each column of computed points ends. In the generator, the wall point is *produced*: the march integrates the mass-flow streamline and the wall is wherever it goes. In the inverse mode the wall point’s *position is given* (it must sit on the prescribed contour at that station, with the contour’s local slope supplied as data), so the wall cell works backwards, exactly as described in Chapter 3: it finds the *foot* of the arriving *characteristic* — the point on the previous column from which the line carrying information to the wall point set out — by a chord search, scanning the stored segments of that column for the one whose data brackets the characteristic condition. It then closes the compatibility relation along the chord from foot to wall point, with the prescribed wall slope supplying the flow direction there [3]. One march column is built per prescribed wall station.

Two consequences shape the rest of the engine. First, an arbitrary wall does not produce a uniform, axial exit, so nothing downstream may assume one: thrust is always evaluated by the wall route (5.2), which needs only the wall pressures and the throat flux. Second, the whole construction remains differentiable — the foot search, the chord closure, the thrust quadrature — so reverse-mode automatic differentiation supplies $dJ/d(\text{wall})$, the sensitivity of thrust to a free-form wall change, at the cost of roughly one extra march.

That gradient is certified by the *envelope identity*, which is the fundamental theorem of calculus used as a test: the average of a derivative over an interval must equal the secant slope across it, $\overline{dJ/da} = [J(a_2) - J(a_1)]/(a_2 - a_1)$. So the test deforms the wall by a smooth localized bump of amplitude a , sweeps a over a range, fits a straight line to the AD gradient samples, and requires the fitted mean slope to match a wide secant of J itself within a derived band. Why not the obvious pointwise comparison of AD against a small-step finite difference? Because the gas properties are read from tables interpolated piecewise-linearly, so the code’s exact derivative carries a microscopic sawtooth at the table-cell scale, and a small-step difference straddles those kinks: that test is ill-posed here. The finding and its diagnosis are in Appendix A.3; the envelope certificate is its resolution and Figure 7.2 its one-picture summary.

7.3 What was built

`validation/a1_wall_march.py`: the prescribed-wall engine (`wall_march`), the station extractor (`stations_from_wall`), the wall-route thrust (`J_wall`), per-column mass flux, the envelope-identity gradient certificate, and rejectors (a wall whose slope data is corrupted must crash the march).

7.4 Results of record

Twin stage, 5/5 PASS. Fed Brick 1’s own contour as a prescribed wall, the new engine reproduces Brick 1’s thrust through a completely different computation — $J_{\text{wall}} = 1.25534 \times 10^8$ N against $J_{\text{exit}} = 1.25521 \times 10^8$ N, a difference of 1.37×10^4 inside a derived band of 1.58×10^5 . The mass flow through the last computed column equals the throat mass flow to 12.3 kg/s (band 152), an independent conservation check the twin comparison does not contain; the minimum Mach number along the wall is 1.20, so the flow stays supersonic everywhere and the method remains applicable; 17,221 cells are Newton-certified, i.e. every local solve converged inside its derived residual band; and the slope-corrupted wall crashes the march — rejected. **Gradient stage, 2/2 PASS** via the envelope identity: $|\text{AD} - \text{secant}| = 29$ against a derived band of 109, and the sensitivity is itself far larger than that band, so the wall demonstrably matters.

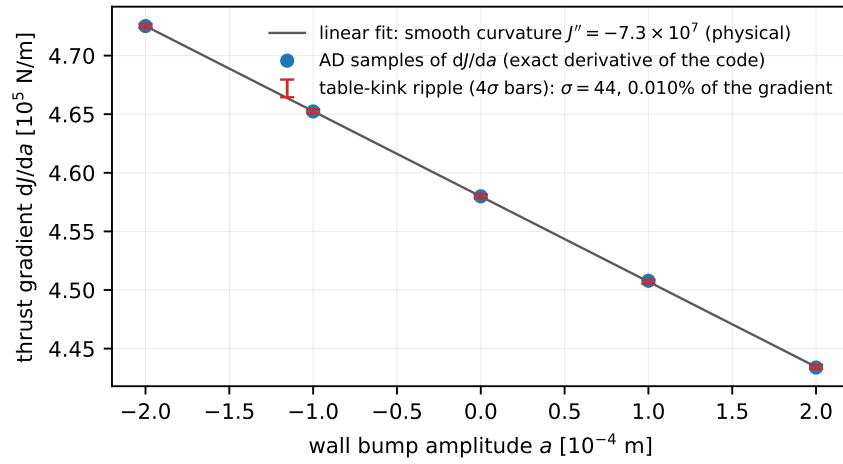


Figure 7.2: Why the gradient is certified by envelope rather than pointwise. Horizontal axis: amplitude a [m] of the smooth bump added to the wall, from indented to bulged. Vertical axis: the thrust sensitivity dJ/da [N/m] returned by automatic differentiation, i.e. newtons of thrust gained per metre of bump. Look at the two things superposed: a straight smooth trend (line; fitted curvature $J'' = -7.3 \times 10^7$ N/m²), which is the physics, and a tiny scatter about it (red 4σ bars; $\sigma = 44$ N/m, 0.010% of the gradient), which is the kink pattern of the piecewise-linear property tables. Both are real properties of the discrete objective — AD reports the code’s exact derivative, sawtooth included — but only the trend is physical, which is why the certificate averages over the ripple instead of sampling it. Full story in Appendix A.3.

Chapter 8

The referee, and Rao's multipliers vs the adjoint

So far the program has only checked itself. This chapter brings in two outside judges: a nozzle design code written by other people in another language, whose whole flowfields we audit against our equations, and the 1958 pencil-and-paper theory of the optimal nozzle, whose predictions we measure on a shape our optimizer did not make. Passing both means the engine solves the right problem, not merely a self-consistent one.

8.1 The referee comes back online

8.1.1 What you need to know first

An *ideal*, or *minimum-length*, nozzle gives a perfectly uniform, purely axial exit flow in the shortest length that can do it (Chapter 2). A *thrust-optimized contour* (Rao's TOC [1]) answers a different question: for a *given* length, which shape produces the most thrust? It is shorter and fuller, its exit flow deliberately non-uniform, and it buys more thrust per kilogram of nozzle. Both shapes here reach the same *area ratio* $\varepsilon = A_e/A_t = 4$: exit area four times throat area, the number saying how much the nozzle expands the gas. Throughout, tolerances are *derived bands* computed from the discretization rather than chosen, and every test carries *rejectors*: corrupted inputs it must fail (Chapter 4; symbols in Table 4.1).

8.1.2 The question

Everything so far checks our code against itself and against closed forms. Where is the *independent* program that can referee whole flowfields?

8.1.3 How it works, in full

GENO [4] is that program: an independent Fortran nozzle designer — separate code base, separate discretization, separate thermodynamic backend — and, by standing decision, never modified by us and never differentiated. As *generator* it produces reference contours and flowfields, in particular the thrust-optimized nozzle Section 8.2 needs: a genuinely optimal shape our optimizer had no hand in. As *referee* its fields are put on trial against *our* equations. Our solver builds a flow on a mesh of intersecting wave lines, enforcing a fixed set of algebraic relations at each mesh point — its *unit process* (Chapter 3); those relations are now evaluated on GENO's own node values, where they must hold within the derived band. The control is a set of deliberately wrong *feet* — the upstream points on the two wave lines from which a mesh point is built — which must *not* hold. Agreement then means the two codes solve the same equations cell by cell, not just that their outputs look alike.

GENO's old binary predated this host's toolchain, so it was rebuilt from source; the regenerated reference case is the type-2 TOC at $\varepsilon = 4$ (874×2885 nodes).

8.1.4 Results of record

- **Cross-code oracle: PASS.** 218/218 sampled cells of the regenerated field satisfy our unit-process relations inside the derived band, with $84\times$ discrimination between correct and deliberately wrong characteristic feet.
- **Brick 1's last SKIP closed.** A Brick-1 test was unrunnable while the referee would not build here. It now runs live, comparing our ideal contour with GENO's station by station: 62/62 samples in band, maximum error 7.6×10^{-9} m.
- **The f_2 positive control landed** (the one declared-pending test of Section 5.2). f_2 combines local flow speed and flow direction, and the theory forces it to be constant along one line of an optimal nozzle — the *terminal characteristic*, defined below. On the genuine Rao-optimal contour it is constant there to a relative spread of 4.0×10^{-3} while along the wall it varies by 0.60 — a $149\times$ separation. The *truncated* reference case correctly showed no constancy: cut short, an optimal contour no longer contains its own control surface.

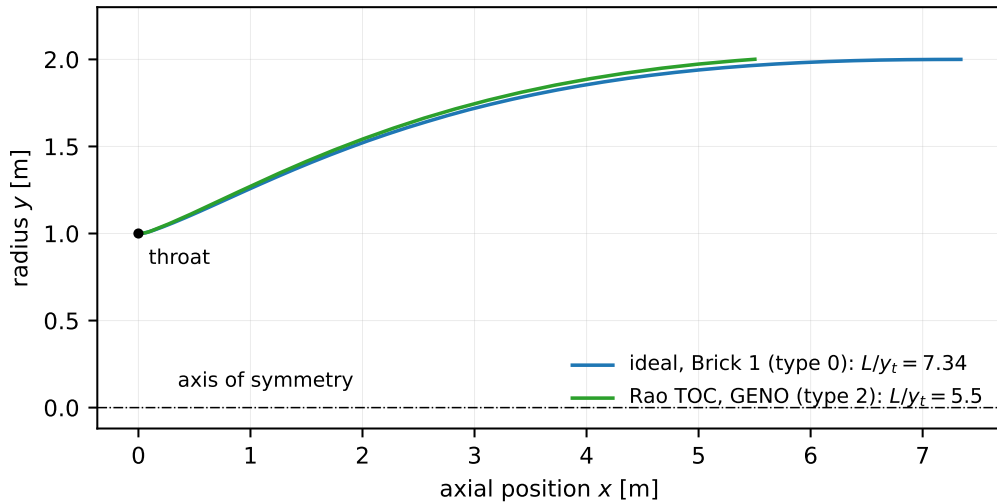


Figure 8.1: The two protagonists, drawn to scale: axial distance x [m] against radius y [m], axis of symmetry at $y = 0$, throat at $x = 0$, $y = y_t = 1$ m (so metres and throat radii coincide here). Both walls reach area ratio $\varepsilon = 4$; look at the *lengths*. Brick 1's ideal (minimum-length) nozzle runs to $L = 7.3409$ m, GENO's Rao thrust-optimized contour stops at 5.5 m: shorter by design (best thrust for a *given* length). An unplanned cross-check: GENO refused a longer TOC request, quoting the ideal length $L = 7.3409$ — exactly our twin's, from a code that never saw ours.

8.2 Rao's multipliers vs our adjoint, on a real optimal contour

8.2.1 The question

Rao solved the optimal-nozzle problem in 1958 with paper and the calculus of variations. This program solves it by *automatic differentiation*: the machine propagates derivatives through every operation the solver performs, giving the exact sensitivity of thrust to every shape parameter — and run backwards (an *adjoint*) one pass returns *all* of them at once, for about the cost of

one extra flow solve. The registry’s oldest open item (its C-O33 in [5]) asks whether the two answers agree, term by term, on a real thrust-optimized contour.

8.2.2 How it works, in full

The meeting ground is GENO’s optimal contour, marched by *our* prescribed-wall engine (Chapter 7). When something is maximized subject to constraints, each constraint enters the solution with a constant that prices it — a *Lagrange multiplier*. Rao’s solution [1] carries two: λ_2 , the price of holding the mass flow fixed, and λ_3 , the price of holding the length fixed. Each is falsifiable, because the theory names a combination of local flow quantities that must equal it at every point of one curve — a *first integral*, a quantity the equations keep constant there. That curve is the *terminal characteristic*: the last C^- wave line of the nozzle, from the axis to the lip, the control surface the calculus of variations selects because thrust and mass flow are evaluated across it. There:

$$f_2 = \frac{W \cos(\theta - \alpha)}{\cos \alpha} = -\lambda_2, \quad 2\pi y \rho W^2 \sin^2 \theta \tan \alpha = -\lambda_3, \quad (8.1)$$

with W the flow speed [m/s] (the q of Table 4.1), θ the flow angle, α the Mach angle, ρ the density, y the radius — both *constant along that curve* if and only if the contour is optimal. The test samples both quantities on GENO’s own terminal-characteristic nodes and demands constancy within derived bands in our field *and* in GENO’s, plus agreement between the codes on the same nodes.

The adjoint side tests *stationarity*, the calculus statement of sitting on a peak: at a maximum the derivative in every *allowed* direction vanishes, so small allowed changes cost nothing to first order. The allowed directions are wall deformations keeping the length and both endpoints fixed — six endpoint-pinned Gaussian bumps (centers at $0.20L \dots 0.85L$). A bump moving the lip probes a direction the theory does not protect (a bigger nozzle always pays a little more) and must give a large gradient; the same six bumps on a deliberately perturbed wall are the positive control. Every AD projection is cross-checked against finite differences, i.e. against recomputing the thrust with the shape nudged by hand.

8.2.3 Results of record

9/9 PASS across two stages (`validation/a1_o33_toc.py`). *Integrals*: our field on GENO’s terminal-characteristic nodes shows f_2 constant with relative spread 4.045×10^{-3} (GENO’s own: 4.04×10^{-3}), and the level matches across codes to 0.235 m/s in 2634.7 m/s — 9×10^{-5} relative (Figure 8.3); λ_3 matches pointwise (its constancy is honestly declared unresolvable at this surface precision: $\lambda_3 \sim \sin^2 \theta$ is small and steep near the axis). *Stationarity*: the six interior gradients sit $1000\times$ below the lip direction and $40\times$ below the perturbed-wall gradients, AD = FD to $\sim 1\%$ throughout, and the optimal contour outscores the perturbed one, $J(\text{TOC}) > J(\text{perturbed})$ (Figure 8.4). One rejector was itself corrected by physics along the way: Appendix A.2. The formal registry discharge (YAML and suite wiring) is a commit-time act on the WSL repository; the numerical content exists.

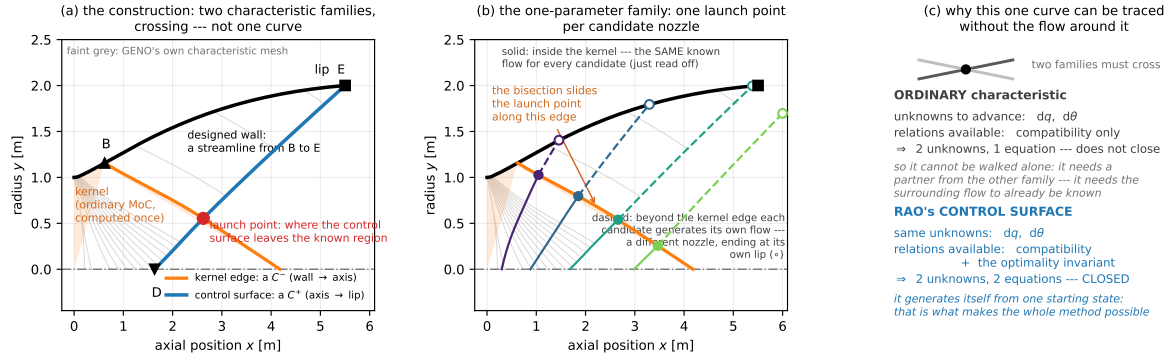


Figure 8.2: How Rao's optimal control surface is actually constructed, drawn on the real thrust-optimized field (axes: axial position and radius in metres; the faint grey lines are the reference code's own characteristic mesh). (a) Two characteristic families that *cross*: the kernel edge (orange), a C^- running from the end of the initial expansion at B down to the axis, and the control surface (blue), a C^+ running from D on the axis up to the lip E. The red dot is the launch point, where the control surface leaves the region the throat already determines. Everything inside the shaded kernel is ordinary marching, computed once. (b) The one-parameter family: each launch point on the kernel edge gives a different candidate nozzle. Solid inside the kernel — the same known flow for every candidate; dashed beyond it, where each generates its own flow and ends at its own lip. The search slides the launch point along that edge. (c) Why this one curve can be traced without knowing the flow around it: an ordinary characteristic has two unknowns to advance and only the compatibility relation to advance them, so it needs a partner from the other family; Rao's optimality condition supplies the missing second relation, and the system closes.

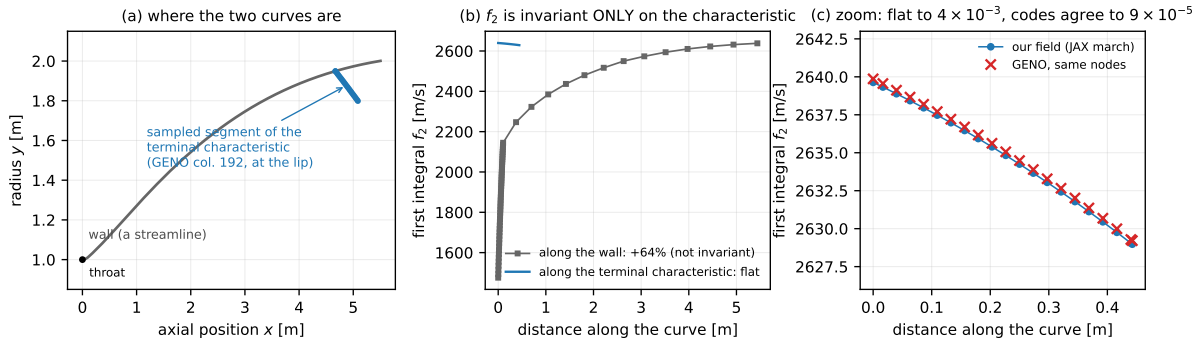


Figure 8.3: The first integral, measured. (a) Where the two curves live: x [m] against y [m] on GENO's thrust-optimized nozzle — the wall (grey) and the sampled segment of the terminal characteristic, its last wave line (blue). (b) f_2 [m/s] against arc length [m] along each curve: it climbs 64% along the wall — no invariant there, and the theory says it must not be — and lies flat along the terminal characteristic. (c) Zoom on that flat line, same units: our field (circles) sits on GENO's (crosses) at the same nodes. Flatness to a relative 4×10^{-3} with cross-code agreement to 9×10^{-5} : Rao's constant $-\lambda_2$ exists in a computed flow, and two unrelated codes find the same value for it.

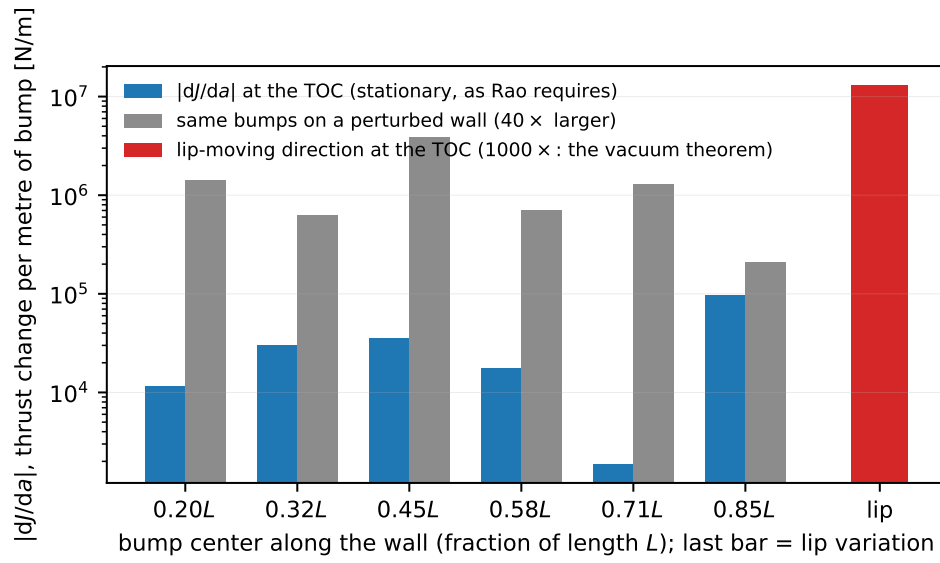


Figure 8.4: Stationarity, on a log scale. Horizontal axis: the wall deformation applied — six Gaussian bumps, labelled by their center as a fraction x/L $[-]$ of the wall length, plus the lip-moving deformation. Vertical axis: $|dJ/da|$ [N/m], thrust change per metre of bump amplitude, logarithmic. On the optimal contour (blue) the gradient sits 40 $\times$ below the same bumps on a deliberately perturbed wall (grey) and 1000 $\times$ below the lip direction (red): flat in the allowed directions, steep in the one direction the theory does not protect. That is what an optimum looks like to the adjoint.

Chapter 9

The cycle layer: the engine becomes an RDE tool

Until now the nozzle had one operating point: one chamber pressure, one chamber temperature, one design. A rotating detonation engine does not offer one. This chapter teaches the optimizer to design against a whole periodic cycle of chamber conditions — and then proves it did so correctly, by making it reproduce, blindly, two answers the theory already knows and a shortcut-taking implementation could not find.

9.1 Cycle-averaged optimization and its oracles

9.1.1 What you need to know first

A *rotating detonation engine* (RDE) burns its propellant in a ring-shaped (annular) chamber around which one or more *detonation waves* — supersonic combustion fronts, a shock with the flame locked to it — run continuously, typically a few thousand times per second, fresh propellant refilling the gap behind each wave. The consequence for the nozzle is the point of this chapter: a nozzle cross-section never sees steady conditions. Once per revolution the pressure jumps sharply as the wave sweeps past, then falls (*blows down*) until the next wave arrives.

What the nozzle responds to is the *stagnation state* (P_0, T_0) : the pressure and temperature the chamber gas would have if brought smoothly to rest, the reservoir condition that fixes everything downstream. In an RDE it is a periodic *family* $(P_0(\xi), T_0(\xi))$ indexed by the *cycle phase* $\xi \in [0, 1]$, one full revolution of the wave; the *phase measure* $d\mu(\xi)$ says how much of the cycle each phase occupies, hence its weight in an average. And *quasi-steady* means the gas crosses the nozzle in far less time than the wave takes to come round again — so at each instant the flow is simply the steady flow of that instant's chamber state. The design variable throughout is the area ratio $\varepsilon = A_e/A_t$, exit area over throat area (Chapter 2).

9.1.2 The question

An RDE hands the nozzle a periodic *family* of chamber states, not one. Can the engine optimize the cycle-averaged thrust — and prove it does so correctly?

9.1.3 How it works, in full

Because the expansion is quasi-steady, the cycle-averaged thrust is simply the phase average of steady thrusts:

$$J_{\text{cycle}}(\varepsilon) = \int_0^1 F(\varepsilon; P_0(\xi), T_0(\xi)) d\mu(\xi). \quad (9.1)$$

The layer implements this *honestly*: the integral is a quadrature over a finite set of phases, and each phase gets its own stagnation state built on the same gas tables, its own full march through the nozzle, and its own thrust. No shortcut exploiting a similarity between phases is coded anywhere — deliberately, because the theory’s hardest test is about what an honest implementation must then reproduce on its own.

That test is the collapse theorem for the fixed bell (its T3) [5]. It says: if only the pressure varies over the cycle and the temperature does not (a *pressure-only* blowdown, fixed T_0), the whole cycle problem collapses to a classical steady design at the mean pressure — so cycle-aware optimization must buy *exactly nothing*. The reason is a similarity: at fixed T_0 and fixed geometry, scaling P_0 scales every pressure in the nozzle by the same factor and moves no wall and no Mach number, so J_{cycle} is the steady J at $\langle P_0 \rangle$, exactly. An engine marching each phase independently has no way to know this — unless it is correct, in which case the zero gain *emerges*. That emergence is the test.

The second stage poses the first mock RDE (user-posed): exponential P_0 and T_0 profiles in phase (peak-to-trough pressure ratio $PR = 10$, temperature ratio $TR = 1.306$, peak 3850 K, five phases spaced uniformly in the logarithm of pressure). With T_0 varying, the collapse is no longer exact, and the question becomes quantitative: how close does the best *one-number* design — an ordinary single-point design at one representative chamber state — come to the blind cycle optimum, and which state is the right one? The theory’s answer — the mean pressure at the *pressure-weighted* temperature $\langle PT \rangle / \langle P \rangle$ — is tested against the log-mean alternative in a derived comparison, and a design made at the peak-pressure phase supplies the deliberately wrong reference.

Building a phase’s stagnation state on tabulated real-gas properties has traps of its own; the three that were met (and the rejectors that now guard them) are Appendix A.4.

9.1.4 Results of record

The zero-gain collapse oracle: 5/5 PASS (validation/a1_cycle_layer.py, stage t3). The optimizer, told nothing about the theorem and spending five full marches per objective evaluation, lands on the design-at-the-mean within its window: the gain it finds over that design is -475 N out of 1.16×10^8 N, against a derived band of 9.6×10^3 N — zero, to the precision the test can resolve. The deliberately wrong peak-pressure design loses 4.8×10^6 N, $500\times$ the band, so a real gain would have been seen. And the similarity shows up live: at fixed geometry the five per-phase walls coincide to 8.9×10^{-16} — pressure scaling does not move the wall, measured at machine zero.

The mock RDE: 6/6 PASS (stage mock). The blind optimum lands within -0.74% in ε of the best one-number design — which is indeed mean pressure at the pressure-weighted temperature $\langle PT \rangle / \langle P \rangle$, beating the log-mean reference in a derived comparison (the theory’s weighted-average lesson, confirmed at engine level); the thrust penalty of that one-number design is 2.5×10^{-6} relative (second order, as predicted), while the peak design loses 4% of J . The practical reading: for this class of cycle one well-chosen state is enough, and the theory says which.

9.2 The brokered compromise, on a meter

9.2.1 The question

The theory’s most quotable claim about RDE nozzles [5] is the averaged corner condition: “no phase satisfies its own corner condition; the cycle average does.” Can that be measured?

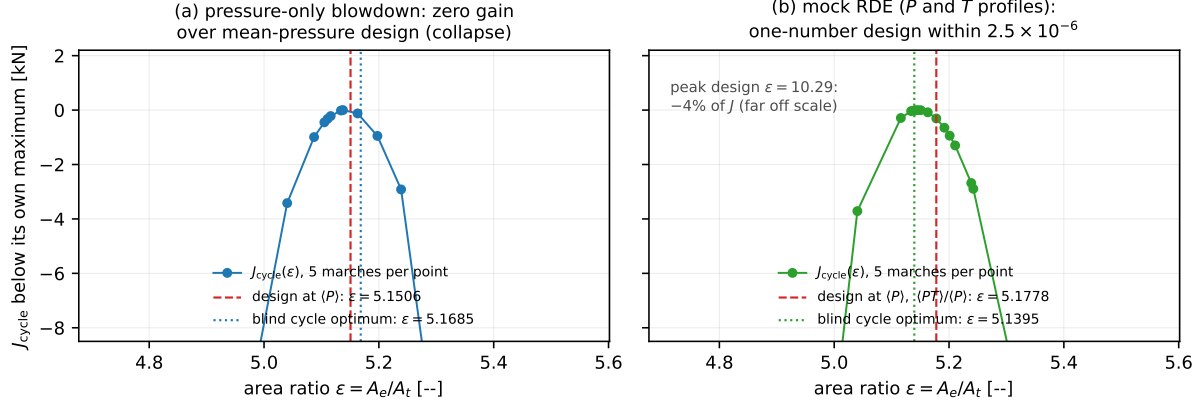


Figure 9.1: The cycle layer, measured. Axes in both panels: area ratio ε [-] (exit over throat area, the design variable) against cycle-averaged thrust of Equation (9.1), plotted as the shortfall below that curve’s own maximum, in kN (zero is the best design on the curve; the absolute level is $J_{\text{cycle}} \approx 116$ MN). Each point costs five full nozzle marches, one per phase; vertical lines mark candidate designs. (a) Pressure-only blowdown: the blind cycle optimum ($\varepsilon = 5.1685$) sits on the classical mean-pressure design ($\varepsilon = 5.1506$) within the derived window — the zero-gain collapse, reproduced by an engine never told it should happen. (b) Mock RDE, pressure *and* temperature varying: the blind optimum ($\varepsilon = 5.1395$) and the one-number design at the pressure-weighted mean state ($\varepsilon = 5.1778$) differ by 0.74% in ε and 2.5×10^{-6} in J ; the peak-pressure design ($\varepsilon = 10.29$) loses 4% and lies far off scale. Note how flat the curves are near their peaks: the story lives in the last digits, which is why the thresholds are derived rather than eyeballed.

9.2.2 How it works, in full

Recall the corner condition of Section 5.2: the exact statement that a nozzle’s wall stops at the right place, whose leading term is the mismatch between the pressure the gas reaches at the exit and the ambient pressure it exhausts into. At the cycle optimum every phase ξ casts a *vote* through that same meter: its own exit-pressure mismatch $R(\xi) = p_e(\xi) - p_a$. A phase with $R > 0$ leaves the nozzle still pushing harder than its surroundings — *underexpanded*, voting “lengthen me”. A phase with $R < 0$ has expanded below ambient — *overexpanded*, voting “shorten me”. The claim is that the optimum satisfies no phase individually: it is a brokered compromise in which the μ -weighted mean of the votes is zero while each vote is not.

The meter reads the votes off the cached cycle runs, and every threshold is derived. The balance band around zero comes from the objective’s own curvature and the width of the window where the optimizer stopped — how far from zero the mean could legitimately sit. Each phase’s vote must then individually exceed that band; otherwise “no phase satisfies its own condition” would be an assertion rather than a measurement.

9.2.3 Results of record

8/8 PASS on both cycles (`validation/a1_cycle_corner.py`, an analysis of the cached cycle runs — 0.4 s). At the mock-RDE optimum the five phases’ votes run from $+7.9 \times 10^5$ down to -5.1×10^5 Pa — monotone in phase, with exactly one sign change, each individually $2.8\text{--}3.2 \times$ the balance band and $14\text{--}90 \times$ the achieved average — while their *mean* sits at zero within the derived band. At the wrong (peak-pressure) design the meter pins at -4.6×10^5 Pa: every phase overexpanded, no compromise at all. A wrong design is visible at a glance.

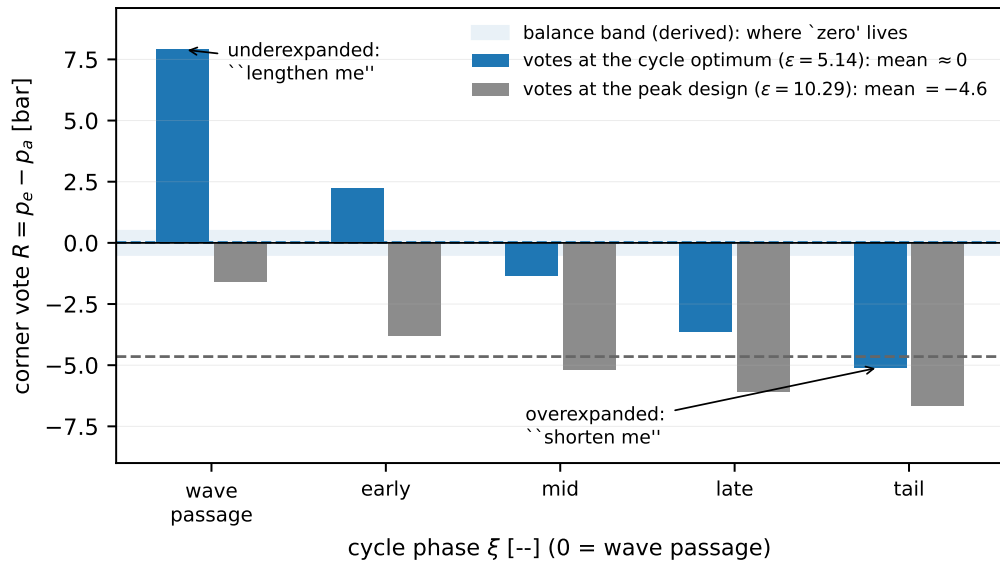


Figure 9.2: The theorem on a dial. Horizontal axis: cycle phase ξ [-], one revolution of the wave from passage to blowdown tail. Vertical axis: each phase's corner vote $p_e - p_a$ [bar], the amount by which its exit pressure exceeds (positive: underexpanded, “lengthen me”) or falls short of (negative: overexpanded, “shorten me”) the ambient pressure. Blue: votes at the cycle optimum — large and of both signs, yet their mean (blue dashed) sits inside the shaded balance band at zero. That is the averaged corner condition, measured: no phase is individually satisfied, the cycle is. Grey: the same votes at the peak-pressure design — every phase overexpanded, the mean (grey dashed) far below zero: a wrong design, at a glance.

Chapter 10

The plug march

Every nozzle in this document so far has been a bell: a solid tube that wraps the gas on all sides. This chapter builds the other kind. A plug nozzle keeps solid metal only *under* the flow and lets the outside of the jet touch the atmosphere directly — one change that buys the nozzle the ability to adapt itself to its surroundings, and costs the solver a boundary it has never had to handle. The step adds that boundary, assembles a complete plug march around it, and certifies the assembly against a flow whose answer is known exactly on paper.

10.1 The question

The deepest results in the RDE nozzle theory are about plugs (the corpus’s plug theorem, its T4) [5] — in particular about what a plug does when the chamber pressure swings over a cycle, which is Chapter 11. None of them can be tested until the engine can march a plug at all. So: what does the solver need that it does not yet have, and can the new assembly be checked against an exact answer rather than only against itself?

10.2 How it works, in full

10.2.1 The topology, and the one new piece of physics

A plug (or *aerospike*) nozzle is a bell turned inside out. In a bell the gas is enclosed by wall all the way to the exit. In a plug the wall lies only below the flow — a central body, the *spike* — and above the flow there is nothing but the surrounding atmosphere. The outer surface of the exhaust is therefore a *free jet edge*: a boundary of the gas that no hardware holds in place.

That is the whole appeal of the shape. Because the outer boundary is free, the jet widens or narrows on its own until its pressure matches the atmosphere. A bell has one fixed exit area, so it is correctly expanded at exactly one ambient pressure and wrong at every other; a plug re-shapes its own jet as the ambient pressure falls with altitude. That is the classical *altitude-compensation* argument for plugs (Chapter 2) — and it is why plugs matter here for a second reason: an RDE swings its chamber pressure every cycle, which the nozzle feels the same way it feels a change of altitude.

The free edge is the one new piece of physics the solver must learn. On a wall, the *position* is known and the pressure comes out as a result; on a free edge it is exactly the reverse. The pressure is prescribed — $p = p_a$, the ambient pressure, because the atmosphere pushes back and nothing else does — and the *position* is what gets solved. In the cell language of Chapter 3, the free-edge cell is fed by the previous edge point (the edge is a *streamline*: a line the flow runs along and no gas crosses) and by the C^+ characteristic rising from the interior point below it in its own column. The prescribed pressure fixes the speed modulus q_{p_a} through the gas tables, so the two remaining unknowns are the new position and the turning angle θ_4 , with

$(u, v) = q_{p_a}(\cos \theta_4, \sin \theta_4)$ (Figure 10.1). The assembly then mirrors the bell march upside down: spike wall below (the inverse-wall cell of Chapter 7, mirrored to the C^- family that a bottom wall receives), free edge above, no axis [2].

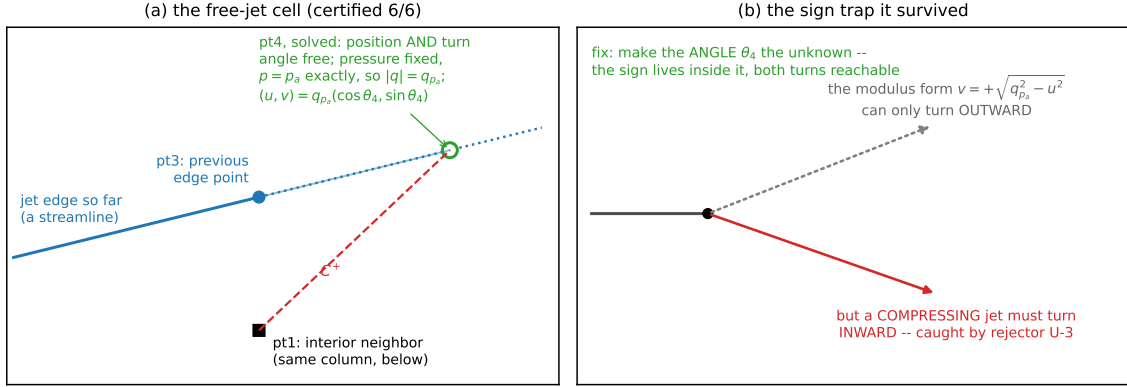


Figure 10.1: The free-boundary cell, drawn in the physical plane: x axial [m] against y radial [m]. (a) The new edge point pt4 is built from two known states — the previous edge point pt3, whose flow direction sets the direction of the edge segment because the edge is a streamline, and the interior neighbour pt1 in the same column, which sends the C^+ characteristic. What to look at is which quantity is held and which floats: the pressure is *imposed*, $p = p_a$ exactly, which fixes the speed modulus q_{p_a} through the tables, while the position (x_4, y_4) and the turning angle θ_4 are solved. That inversion — the opposite of a wall cell, where position is given and pressure comes out — is what makes a plug a plug. (b) The same cell with the sign trap it survived on the way to certification: writing the boundary condition as $v = +\sqrt{q_{p_a}^2 - u^2}$ admits only outward turns (Appendix A.5).

10.2.2 Why the lip fan is constructed, not marched

At the *cowl lip* — the sharp outer edge of the channel that feeds the plug — the gas leaves the internal duct and turns around a corner into the open. A supersonic gas turning away from itself expands through a *Prandtl–Meyer fan*: a continuous sheaf of straight rays, each carrying one intermediate state, across which the flow accelerates, its pressure falls, and its direction rotates smoothly from the incoming angle to the outgoing one. When the corner is sharp, every ray starts from the same geometric point and the fan is *centered* on it. That point is *singular*: all the intermediate states coexist there at once, so the flow has no single value at the lip and its gradients are infinite (Figure 10.2).

This violates exactly the assumptions an ordinary cell rests on. A cell whose segment spans the fan closes its compatibility relation with a midpoint average of two states that differ by the whole turn — an order-one error, not a second-order one — and near the lip all the C^- characteristics meet at one point, so the position crossing degenerates. Measured: a certification residual of 8.9×10^9 when an early build tried exactly this.

The cure costs nothing, because a centered fan is self-similar: the state on each ray depends only on the ray angle and is given exactly by the Prandtl–Meyer function. So the corner is never marched, it is *constructed* — seed the fan as data, then resume ordinary cells between rays, where the flow is smooth again. Brick 1 does precisely this at its sharp throat corner (the initial expansion fan); the oracle below goes one step further and places the start line downstream of the lip with the exact fan field written on it, so the singular point lives in the data and never inside a cell.

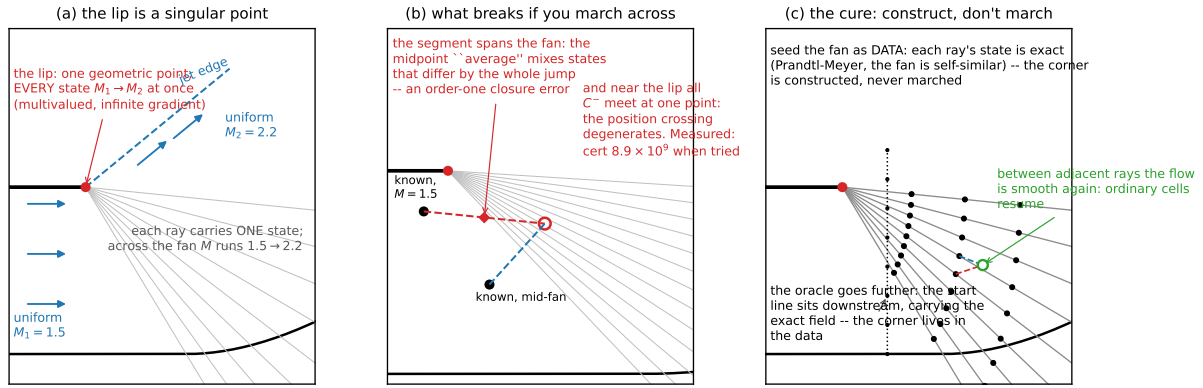


Figure 10.2: Why the lip fan cannot be marched through. All panels are the physical plane, x [m] against y [m]. (a) The lip as a singular point: a uniform stream at M_1 and a uniform stream at M_2 meet across a centered fan whose every ray emanates from one geometric point, so at the lip every state between M_1 and M_2 coexists and the gradient is infinite. (b) An ordinary cell asked to step across it: its segment spans the fan, so the midpoint closure averages two states separated by the whole turn, and the C^- crossings degenerate as they converge on the lip — measured, a certification residual of 8.9×10^9 . (c) The cure: each ray is seeded with its exact Prandtl-Meyer state and ordinary cells resume between rays; the oracle puts the start line downstream, carrying the exact field. What this proves is that the singularity is handled by construction, not by refinement — no mesh is fine enough to march through a point where the solution is multivalued.

10.2.3 The march: columns, growth at the edge, consumption at the wall

Each column is swept *bottom-up*: first the wall point at the prescribed station, then the interior points, each fed by the C^+ from the point just solved below it and the C^- from its partner on the previous column, and finally the free-edge point on top, fed from its own column (why it must be its own column is a lesson the oracle taught: Appendix A.6.1).

The wall cell finds its foot — the point on the previous column where the characteristic arriving at the wall started — by a *multi-column search*: it scans the whole previous column, stepping further columns back if needed, for the segment whose data brackets its arriving C^- (the bracketing function is positive at the previous wall point and decreases with row height, so the first sign change marks the foot segment). This lets the march run at fine prescribed stations with short compatibility chords; the earlier scheme, which confined the foot to the first segment of the adjacent column, had to stretch its columns and paid for it in accuracy (Appendix A.6.3).

Row bookkeeping closes the topology, one characteristic at a time, at both boundaries. At the *edge*, each column's free-edge point becomes a new top row: the widening jet gains one crossing per column (the mirrored $j_2 \pm 1$ of the bell march; what happens without it is Appendix A.6, lesson 2). At the *wall*, rows are instead *consumed*: the incoming C^- family terminates on a solid wall, so any previous-column rows lying below the new wall point's foot have already met the wall between stations and must not be prolonged — the new column's interiors are partnered with the previous rows *from the foot up*, re-indexed. Prolonging them was the last defect the oracle caught: the terminated characteristics kept marching as ghost rows below the spike — for such a row the only crossing Newton can still find is the backward one, a valid root of the equations on the wrong branch — and the mass check exposed the ghost limb at +9%. With consumption, mass closes to 0.026%.

10.2.4 The oracle

The march is certified against a flow whose *entire field* is known in closed form, so the marched answer can be compared with the truth rather than with itself. The flow is a planar corner jet: uniform $M_1 = 1.5$, a centered fan at the lip, uniform $M_2 = 2.2$ turned by $\Delta\nu = 23.9^\circ$, with the ambient set to $p_a = p(M_2)$. The prescribed spike is the exact streamline of that field, obtained by fine-step RK4 integration — an independent reference, not another method-of-characteristics run — and the start line carries the exact field on a vertical cut.

The marched solution must then reproduce four things, each within a tolerance derived from the scheme's own convergence order rather than chosen by hand (Chapter 4): the edge's asymptotic angle and speed (P-1), the wall pressure distribution station by station (P-2), mass conservation through the final column (P-3), and a *momentum balance* (P-4). The last one is worth spelling out. Draw a closed *control surface* around the gas — an imaginary boundary whose faces are the start line, the final column, the spike surface and the free edge — and require that the momentum flowing out equals the momentum flowing in plus the push the pressures exert on the faces, every pressure measured relative to ambient. On the free edge the pressure *is* ambient, so measured that way the free face pushes exactly zero: the plug's defining property, here tested rather than assumed. Alongside these, a deliberately corrupted ambient (R-1) must leave the bands — a check that the checks can fail. Every cell is Newton-certified.

10.2.5 The axisymmetric stage

For the axisymmetric plug on the real-gas tables no closed form exists, so that stage certifies by *internal* closures — conservation laws the solution must satisfy whatever the answer is. The start-line data is a tables-consistent corner fan ($d\theta = \sqrt{M^2 - 1} dq/q$ integrated on the tables' own $M(q)$), the prescribed spike is a streamline of that data, and the checks are: Newton certification (A-1); mass conservation with the $2\pi y$ flux weight of an axisymmetric ring (A-2); the same ambient-gauge momentum balance (A-3); a *source-alive* control — the same data marched with the axisymmetric source term switched off ($\delta = 0$, planar) must differ by more than the axisymmetric pair's own tolerance band, which guards against an axisymmetric mode that is silently running planar (A-4); and a corrupted-spike rejector (A-5).

10.3 What was built

`validation/a1_freejet_unit.py`: the free-jet cell alone, against unit known answers.
`validation/a1_plug_march.py`: the assembly — bottom-up columns at prescribed stations, the mirrored inverse-wall cell, the multi-column foot search, row growth at the edge and row consumption at the wall — plus the conservation instruments (`col_fluxes`, `wall_push_poly`), the closed-form oracle with its rejector (`STAGE=oracle`), and the axisymmetric internal-closure stage (`STAGE=axi`).

10.4 Results of record

The free-jet cell: 6/6 PASS. A uniform jet already at ambient pressure is an exact fixed point; a turned jet is preserved exactly in planar flow — in axisymmetric flow the source term legitimately changes a turned uniform state, a physics point the first test tripped over; a compressing edge turns inward; and the Prandtl–Meyer corner known answer lands at machine precision — edge speed vs. closed form 1.9×10^{-9} , edge angle equal to the exact turn angle to 1.9×10^{-16} .

The assembly vs the oracle: 6/6 PASS (66 s, stations 41/81, start rows 21/41). Every cell Newton-certified — worst residual 0.019 (coarse, 1253 cells) and 0.017 (fine, 4870 cells) of the allowed tolerance. P-1: the marched edge's tail angle is 0.41809 vs. the exact $\Delta\nu = 0.41777$ — $|d| = 3.2 \times 10^{-4}$ against a band of 4.5×10^{-3} , with the edge speed equal to the closed form

to all printed digits. P-2: 98% of wall stations inside the derived band, maximum deviation $9.8 \times 10^{-3} p_a$. P-3: mass through the final column 8869.98 vs. start-line 8867.65 — $|d| = 2.33$ (0.026%) against a band of 26. P-4: momentum closure 6.9×10^3 on $F_{in} = 2.0 \times 10^7$ (3.4×10^{-4} relative; band 7.2×10^4 ; coarse/fine ratio 3.6, confirming the scheme's second order), with the free edge contributing zero by construction. R-1: a 1% corruption of the ambient moves the edge angle $20\times$ outside the P-1 band — rejected. Figure 10.3 shows the certified march; the road here — four structural lessons, each measured — is Appendix A.6.

The axisymmetric stage: 5/5 PASS (39 s, stations 33/65, start rows 17/33, NASA real-gas tables, tables-fan turn 0.4395 rad). A-1: all cells Newton-certified, worst residuals 0.017/0.022. A-2: mass with the $2\pi y$ weight, $|d| = 505$ on 1.085×10^5 (0.47%) against a band of 986. A-3: momentum closure -2.4×10^6 on $F_{in} = 2.4 \times 10^8$ (1.0×10^{-2} relative, inside its band). A-4: the axisymmetric source is alive — the wall speed at the last station differs from the planar run by 156 m/s, $9\times$ the band. A-5: a corrupted spike slope breaks the momentum closure at $5\times$ the band — rejected.

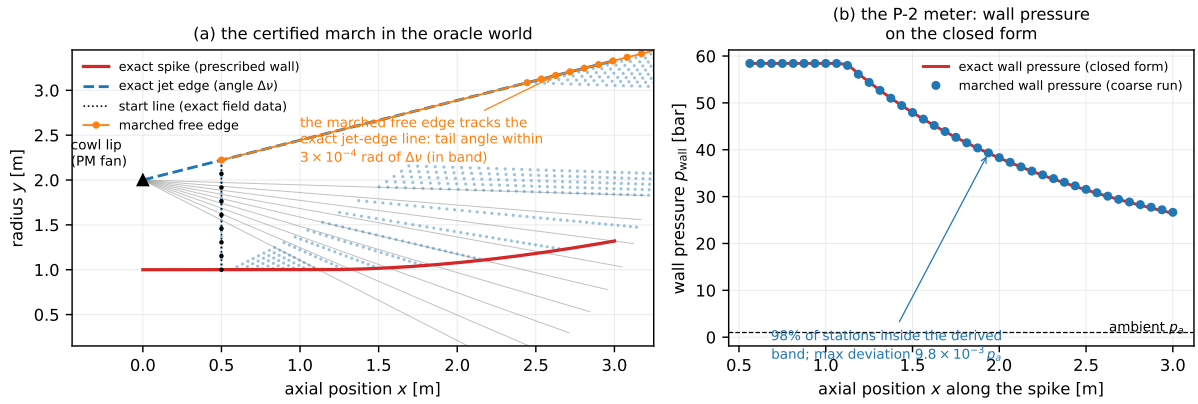


Figure 10.3: The certified march against the closed-form oracle. (a) The physical plane, x [m] against y [m]: the exact world — spike (red), exact jet edge (blue dashed), lip fan rays (grey), start line (dotted) — overlaid with the marched characteristic mesh (light dots) and the marched free edge (orange). Look at how the orange edge tracks the blue line: told only the pressure on that surface, the solver found where the surface is. (b) Wall pressure along the spike, x [m] against p_w [bar]: marched values against the closed form, with the derived tolerance band — 98% of stations inside it, worst deviation $9.8 \times 10^{-3} p_a$. The two panels together are the certificate: the free surface and the solid surface are both right. Compare with the state before the fixes, Figure A.3.

Chapter 11

The truncated plug under the cycle

Chapter 10 can march one plug at one chamber condition. A rotating detonation engine never offers one condition: its detonation wave sweeps the annulus and a fixed cross-section sees the chamber pressure spike and blow down, over and over. This chapter cuts the spike short — real plugs are always truncated, because the long thin tail weighs a great deal and produces almost no thrust — and asks where to cut when the chamber pressure swings like that. The answer turns out not to be “design for the average condition”, and the difference is measurable.

11.1 The question

Two nozzle types behave in opposite ways over a cycle. The fixed bell *averages*: its cycle optimum is the design for the single mean-pressure condition, which the engine certified in Chapter 9. The full plug instead *maximizes*: every phase’s optimum contains every longer plug, so the ideal plug is sized at the strongest phase. The *truncated* plug is where both mechanisms fail at once — “the first genuinely averaged shape problem” [5]. Can the engine measure all three claims: the ordering, its break, and the averaged optimality condition that replaces it?

11.2 How it works, in full

11.2.1 What the theory claims, and what it assumes

Two pieces of vocabulary first. A *cycle phase* $\xi \in [0, 1]$ labels where you are in the repeating chamber history — ξ near the wave passage is the high-pressure part, ξ near the end of the blowdown is the low-pressure part (Chapter 9); the cycle mean μ is the time-like average over those phases. And a plug is *truncated* at length l when the spike is cut off there and closed with a flat base: the whole design question is what l should be.

The corpus’s plug result (its T-T4) is proved under an *ideal-adaptation* closure — an assumption made to keep the proof tractable: that downstream of each phase’s full-expansion point the wall pressure clamps exactly to the ambient p_a , and upstream of it scales with the chamber pressure. Under that assumption the thrust of a given phase, $F(l; \xi)$, never decreases as the plug is extended and becomes exactly constant beyond a phase-dependent length $l(\xi)$, with $l(\xi)$ larger for stronger phases. Each phase’s set of optimal lengths is then a half-line $[l(\xi), \infty)$, and these half-lines are *nested*: the strongest phase’s half-line sits inside all the others. Nesting is what makes the problem collapse — one length is optimal for everybody at once, so the best average equals the average of the bests, $\max_{\Sigma} \int F d\mu = \int \max F d\mu$, attained by the untruncated plug sized at the peak phase.

The sharpness caveat attached to that theorem says what destroys the nesting: a cap on length, a base-pressure model, or adaptation that is not ideal. Once nesting is gone the equality becomes a strict inequality, $\max \int < \int \max$, and the optimum instead satisfies an *averaged corner*

condition at the truncation plane — the per-phase wall-pressure votes there must balance in the cycle mean. What the engine adds is precisely the quantity the closure assumes away: the real marched wall pressure does not clamp. Downstream of adaptation it undershoots and overshoots p_a , and the size of that departure *is* the adaptation break, measurable phase by phase.

11.2.2 Averaging the problem is not averaging the condition

The cycle mean μ is a time-like average over the phases, and it is tempting to conclude that the “ μ -averaged corner condition” is just the mean-pressure design in disguise. It is not, and the distinction is the whole content of the dichotomy. *Mean-pressure design* means: build the single steady flow of the averaged *state* — the phase whose chamber pressure is $\bar{P}_0$ — and optimize that one flow. For the fixed bell this is legitimate, because the bell’s flow and thrust are *linear* in the phase pressure: the scaling law (Lemma A) says the whole field scales with P_0 on a fixed geometry, with the ambient entering only additively through the pressure gauge. For a linear problem the average of the solutions is the solution of the average — average-then-solve equals solve-then-average. That is exactly why the bell collapses.

The *averaged corner condition* instead solves each phase as its *own nonlinear flow*, lets each cast its own vote $p_w(l^*; \xi) - p_a$ at the truncation plane — positive means that phase is still pushing there and wants more plug, negative means it is being dragged and wants less — and averages the *optimality condition*: $\int (p_w(l^*; \xi) - p_a) d\mu = 0$. The two coincide only if the map from phase to vote is affine — and the plug breaks that affinity through its free edge: the ambient is *shared and fixed*, it does not scale with the phase, so each phase’s wall pressure saturates toward the *same* p_a at its *own* adaptation point. Mean-of-votes is not vote-at-the-mean. The suite below measures the difference directly: the mean-pressure design gives $l = 1.280$ m, the averaged condition puts the optimum at $l^* = 2.103$ m — nowhere near. In one line: the bell result averages the *problem*; the plug result averages the *condition*, and averaging commutes with linear problems only.

11.2.3 One march per phase covers every truncation

In supersonic flow, disturbances cannot travel upstream: what happens at a point is set entirely by a bounded region behind it, its *domain of dependence*. On a *fixed* spike, truncating at l removes wall only downstream of l , so nothing at $x < l$ can feel the cut. One full-spike march per phase therefore yields the whole family of truncations in closed form,

$$J(l; \xi) = F_{\text{in}}(\xi) + \int_{x_0}^l (p_w(x; \xi) - p_a) 2\pi y_w (-dy_w), \quad (11.1)$$

where every pressure is measured relative to ambient, so the free edge contributes zero (the pressure on it *is* p_a) and a base plane at $p_b = p_a$ contributes zero as well. Differentiating in l gives the marginal condition $dJ/dl = 0$, i.e. $p_w(l) = p_a$ — the plug corner condition at the truncation plane: extend the plug exactly as far as its wall is still pushing harder than the atmosphere. Its cycle average over phases is the averaged corner condition above. The five-phase truncation study therefore costs five marches, and every check below reads the cached wall-pressure curves.

11.2.4 The world

Axisymmetric, real-gas NASA tables, one fixed piece of hardware marched by every phase. Cowl lip at $(0, 2)$; annular flow arrives at $M_i = 1.2$ aimed toward the axis ($\theta_i = -25^\circ$); the lip fan — the corner expansion of Chapter 10, built per phase and consistent with the tables — expands it down to the shared ambient. The ambient is posed so that the fan turn is about $|\theta_i|$, i.e. the flow leaves roughly axially ($\theta_E \approx 0$): the classic plug design condition, which makes the spike —

one streamline of the mean phase’s fan field — genuinely *descending* toward the axis. The start line poses the same $(M, \theta)(y)$ profile for every phase with each phase’s own thermodynamics: exact scaling-law data for the pressure-only cycle, and the natural “same injector Mach profile” posing for the mock cycle. Each phase runs through the certified plug march of Chapter 10 (prescribed wall, multi-column foot search, row bookkeeping); the cycle itself is the certified construction of Chapter 9 (five log-uniform phases, pressure ratio $PR = 10$, mock temperature ratio $TR = 1.306$).

11.3 What was built

`validation/a1_plug_cycle.py`: the world builder (per-phase tables-consistent fan, mean-field spike streamline), the per-phase march kits with disk cache, the closed-form truncation family (11.1), and two stages — `t3` (pressure-only cycle: the wall-level scaling law and the truncation-level anti-collapse) and `mock` (P and T profiles: the ordering of the per-phase optima, the measured adaptation break, the averaged corner votes, the strict $\int \max > \max \int$ gap, the peak-design penalty) — plus a corrupted-ambient rejector.

11.4 Results of record

Pressure-only cycle (the plug anti-collapse): 5/5 PASS. All phase marches Newton-certified. The wall-level scaling law appears live exactly where it must and only there: on the part of the wall still shielded from the ambient — upstream of the first edge-arrival, $x < 0.81$, found by tracing the edge’s C^- back to the wall — the scaled wall pressures p_w/P_0 collapse to one curve to 4.7×10^{-15} , machine zero. Beyond that station the collapse breaks, and that break *is* the adaptation mechanism, cleanly localized. The truncations then measure the dichotomy: the cycle-optimal $l^* = 2.103$ sits far *above* the mean-pressure design ($l = 1.280$) and just below the peak design ($l = 2.336$) — the plug does *not* average, it shifts toward the peak, held short of it only by the real adaptation break. The per-phase optima $l(\xi) = 2.336, 2.061, 1.125, 1.047, 1.006$ increase with the phase pressure: the nesting *order* of the plug theorem, live.

Mock RDE (P and T profiles), the genuine average: 9/9 PASS. The same ordering appears again ($l(\xi) = 2.336 \dots 1.012$). The non-ideal adaptation is *measured*: each interior phase loses thrust past its own $l(\xi)$ (relative losses 1.6×10^{-4} to 2.0×10^{-3}) — the ideal clamp does not hold in the real flow, and that loss is exactly the quantity that breaks the ideal nesting. Consequently the strict inequality of the sharpness caveat is resolved: $\int \max - \max \int = 1.07 \times 10^4$ N (2.3×10^{-4} of J), beyond its derived band; the penalty for designing at the peak is 2.2×10^{-4} of J , also resolved. At the blind cycle optimum the averaged corner condition holds: the vote mean is $+1.8 \times 10^4$ Pa against a balance band of 3.1×10^5 , while individual votes reach $\pm 1.4 \times 10^6$ — and the optimum is a strict interior compromise (the per-phase optima spread over 1.32 in l ; no extreme phase owns l^*). A corrupted ambient moves the vote mean $2.3\times$ outside the band — rejected. The road here — two posing lessons and two checks the theory itself rejected — is Appendix A.7.

A measured lesson that is new physics, not a bug. The bell’s pointwise vote structure — monotone in phase, exactly one sign change (Chapter 9) — does *not* carry over to the plug. Downstream of adaptation the wall pressure *oscillates* around p_a , because the lip fan reflects off the free edge and comes back down onto the wall. Pointwise votes therefore sample an oscillation, and the wave-passage phase can vote negative at a length below its own optimum. Only the averaged balance and the compromise structure survive — which is exactly, and only, what the sharpness caveat claims.

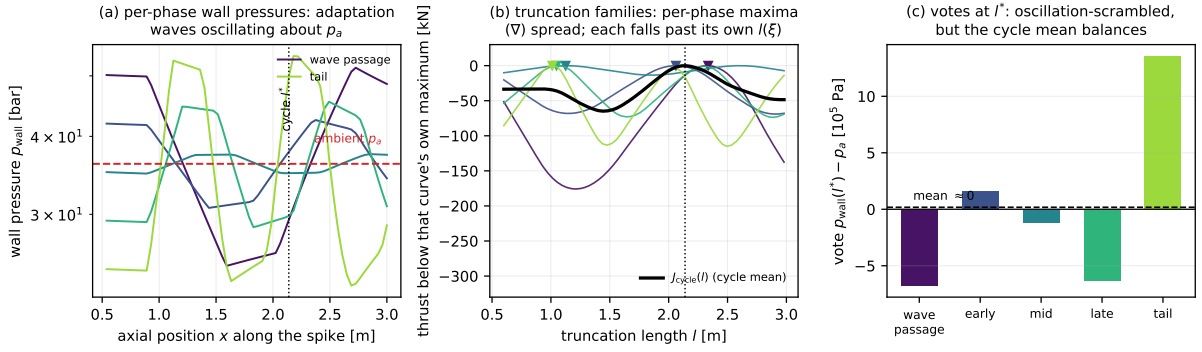


Figure 11.1: The truncated plug under the cycle, measured. (a) Wall pressure along the fixed spike for each phase: x [m] against p_w [bar], with the shared ambient p_a drawn as a horizontal line. Look downstream of the shielded region: the curves do not clamp to p_a , they *oscillate* about it — the lip fan reflecting off the free edge back onto the wall. That oscillation is the real, non-ideal adaptation the theory’s closure assumes away. (b) The truncation families of Equation (11.1): truncation length l [m] against each curve’s thrust deviation from its own maximum, $J(l; \xi) - \max_l J(l; \xi)$, in kN, so zero means “at this phase’s own optimum”. The per-phase maxima (∇) spread from $l = 1.01$ to 2.34 m in order of phase pressure, and each curve *falls* past its own maximum — the measured break of the ideal nesting, which would have kept them flat. The cycle mean $J_{\text{cycle}}(l)$ (black) peaks at a compromise l^* that no single phase owns. (c) The votes at that optimum: cycle phase ξ [–] against $p_w(l^*; \xi) - p_a$ [bar]. They are scrambled in sign order by the adaptation oscillation — the bell’s monotone vote structure does not transfer — yet their cycle mean (dashed) balances at zero inside the derived band. That is the whole claim: not that any phase is satisfied, but that their average is.

Chapter 12

The GENO full-field plug twin

Two programs written independently, in different languages, checking each other on one physical problem is a stronger test than anything either can run alone: a shared answer cannot come from a shared mistake. This chapter runs that experiment on a *plug* (aerospike) nozzle — one whose solid wall, the *spike*, sits under the exhaust while the outside of the jet is bounded by nothing but the surrounding atmosphere. The independent Fortran design code GENO [4] designs such a spike; our solver takes that shape as an ordinary given wall, recomputes the whole flow around it, and the two flowfields are confronted number by number. The step delivers the program’s strongest external check — and one honest finding.

12.1 The question

The two-constraint GENO session (Chapter 15, Section 15.2) left one named residual: the *full-field* comparison. Not a handful of summary quantities, but our certified plug march run on the spike contour GENO’s RaoPlug module now cleanly produces, compared field against field — wall pressure station by station, velocities at GENO’s own interior points, and the integrated mass and thrust. This is the plug counterpart of the bell nozzle’s cross-code check of Chapter 8.

12.2 How it works, in full

12.2.1 What you need to know first

The solver advances a supersonic flow along the two families of *characteristics* — the lines along which a disturbance can travel in supersonic flow, tilted by the Mach angle on either side of the flow direction (Chapter 3; symbols in Table 4.1). Three more words are needed here.

A *start line* (or Cauchy cut, after the initial-value problem it poses) is a line drawn across the flow on which the complete state — position, velocity, pressure — is supplied as data. Given a start line and the walls, the equations determine everything downstream of it and nothing upstream. Ours is a vertical cut across GENO’s finished field.

The *initial-value line* (IVL) is the last characteristic of GENO’s own construction — the outer edge of the region GENO computes. Below it is GENO’s design field; above it GENO has nothing to say.

Truncation appears here in two unrelated standard senses, and the chapter needs both: cutting the spike short at a finite length (hardware), and the error a finite-step scheme makes because its steps are not infinitesimal (numerics). The finding below is about the second.

12.2.2 Two independent constructions

GENO builds the spike *backward*: it rotates the expansion fan at the cowl lip in fixed angular steps (0.05°), integrates one characteristic per fan member with its own cells, places each wall

point by requiring that the mass flowing inside that characteristic equal the mass the throat passes (the mass termination, GENO’s N-65), and cuts the spike at the point D where that balance is met.

Our engine knows nothing of that construction. It is a general axisymmetric march — interior cells, prescribed-wall cells with the multi-column foot search, the free-jet edge, each certified in Chapter 10 — and it treats GENO’s contour as an ordinary given wall, marching *forward* from a start line. Agreement is therefore evidence about the shared continuum problem, not about shared code.

12.2.3 The posing: all data GENO’s own

Nothing analytic is assumed. Even the gas is identified *from GENO’s field*: the ratio of specific heats $\gamma = 5.34782609/4.34782609$ (the recipe GENO’s own thermo backend implements, a gas of constant specific heat), the gas constant R_g from $p/(\rho T)$ at its points (scatter 8.7×10^{-9}), and the field’s own effective isentrope. That last one matters: GENO’s field actually sits at a chamber pressure of $p_0 = 6.008172 \times 10^6$ Pa, *not* the nominal 6×10^6 — a uniform offset of its construction, spread 1.1×10^{-4} — and posing the nominal value would have manufactured a difference that is not there.

The start line is a vertical cut at $x_0 = 0.15$ through GENO’s own first-phase characteristics; each crossing is interpolated on that curve’s own points and cross-checked against an independent two-dimensional interpolation of the same field, agreeing to 10^{-5} . The wall is GENO’s contour, with slopes $\tan \theta_w$ measured from it (its tangency is consistent to 1.6×10^{-5}).

Above the initial-value line lies a region GENO does not compute, and our march needs something there, so a free-jet strip is posed, blending continuously from the state at the IVL crossing to the state on the last fan member. It cannot contaminate the comparison — a fact about supersonic flow, not a hope, since each point is influenced only by what lies upstream inside its cone of dependence: every characteristic arriving at the wall ahead of D leaves the start line *below* the IVL, and the IVL itself lands exactly at D , the end of the wall. It was verified anyway: replacing the blended strip by a uniform one changes the wall by nothing.

12.2.4 Thresholds: declared, with provenance

Everywhere else in this program the pass band is *derived*: the same computation is run at two resolutions, the change measures the scheme’s own error, and the band is that change inflated by a safety factor (Chapter 4). Here the levels are instead *declared* — fixed in advance, with their provenance — deliberately, for the reason that is this step’s central finding below: the measured difference between the two codes sits *above* what either code’s own step refinement suggests, so a refinement-derived band would be the wrong ruler.

The declared levels come from the cross-code agreement already established — an independent integrator of the design curve, built in the two-constraint session, matched GENO to three or four significant digits — re-measured here on the full field, with at least a factor of two of margin on every statistic: wall pressure 3×10^{-3} (measured median 1.9×10^{-3}), settled field $q \ 10^{-3} / \theta \ 1.5 \times 10^{-3}$ (measured 95th percentile $4.5 \times 10^{-4} / 5.3 \times 10^{-4}$), integrals through the shared boundary 2.5×10^{-3} (measured 1.2×10^{-3}). What keeps declared numbers honest are the *rejectors* — deliberately corrupted runs the suite must fail (Chapter 4): they prove the instrument resolves defects an order of magnitude above the gap it is measuring. One check keeps fully derived bands: the gas identification is judged against GENO’s own measured self-consistency floors (total enthalpy 2.0×10^{-5} , entropy $4.7 \times 10^{-6}R$, total pressure 1.1×10^{-4}).

12.3 What was built

`validation/a1_gen0_plug_twin.py`: the reference extractor (GENO run directory $\rightarrow$ `_geno_twin/ref.npz`: wall, initial-value line, the 131,589 first-phase field points, gas identification, floors), the start-line world builder, the calibrated flux quadrature — along GENO’s full initial-value line it reproduces GENO’s printed mass flow $\dot{m}$ to 2.4×10^{-7} and thrust F to 2.3×10^{-7} , settling a question of definition on the way: GENO’s F is the absolute-pressure momentum integral — plus the six agreement checks G-1–G-6 and the two rejectors R-1/R-2.

12.4 Results of record: 8/8 PASS

G-1, the gas: our tables reproduce GENO’s pressure, temperature and density at its own points from the local speed alone, within GENO’s measured floors (worst 9.9×10^{-5} in p). G-2: all 25,034 cells Newton-certified at both resolutions. G-3, the wall: pressure within 3×10^{-3} at 100% of stations (median 1.9×10^{-3} , worst 2.5×10^{-3}). G-4, the interior: past the start transient, 97.7% of GENO’s own points agree within (q 10^{-3} , θ 1.5×10^{-3}); medians 3.8×10^{-4} and 7.9×10^{-5} . G-5/G-6: mass and thrust through the shared segment of the initial-value line agree to 1.2×10^{-3} and 1.1×10^{-3} . Both rejectors fire: a gas falsified by 1% ($\gamma \times 1.01$) collapses the wall agreement to 31% of stations, a contour scaled by $\times 1.005$ collapses it to 0%.

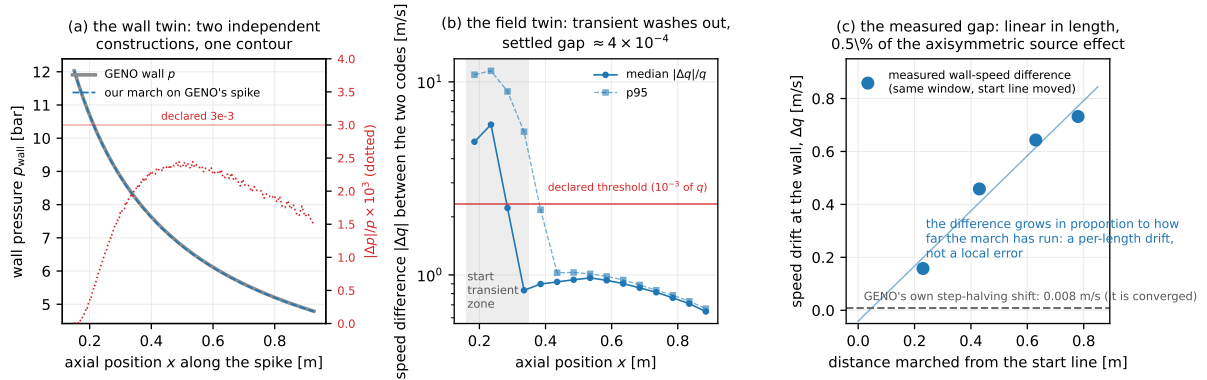


Figure 12.1: The GENO full-field plug twin. (a) Wall pressure along the spike: axial coordinate x [m] horizontally, pressure on the spike surface [bar] vertically. GENO’s own wall (solid) and our forward march on the same contour (dashed) lie on top of each other; the relative deviation (dotted, right axis, in units of 10^{-3}) stays under the declared 3×10^{-3} at every station — two codes, one contour, three to four digits. (b) The same comparison inside the flow: at GENO’s own field points, the difference in flow speed between the two codes, $|\Delta q|$ [m/s], against x [m] (median and 95th percentile per band). Look at the left end: the start transient — our march ingesting a start line drawn through the strong flow curvature near the lip — washes out by $x \approx 0.35$ m, leaving a settled gap of about 4×10^{-4} of the local speed. (c) What that settled gap is: the length actually marched [m], varied by moving the start line, against the resulting speed difference at the wall Δq [m/s]. The points fall on a straight line through the origin ($\approx 4 \times 10^{-4}$ of q per unit length), while halving GENO’s own steps moves GENO’s wall by only 3.5×10^{-6} — so this is no local mistake in either code but a slow drift accumulating with distance marched between two individually converged solvers, worth 0.5% of the axisymmetric source term’s own contribution.

12.5 The finding: a measured, attributed-open cross-code gap

The twin certifies three-to-four-digit agreement — and the residual difference is itself a characterized result, not a leftover. It has two parts. Near the start line, a transient: at mid heights the flow angle differs by up to 1.1×10^{-2} rad, decaying by $x \approx 0.35$, unchanged when the resolution is changed, and moving when the start line moves — the signature of ingesting a start line drawn through the strong flow curvature near the lip. It washes out. What remains is a settled *uniform drift*: our speed runs $+4 \times 10^{-4}$ relative high across the whole column, and that excess grows linearly with the distance marched (Figure 12.1c).

The forensics excluded, by measurement, every accessible cause. *The gas*: both sides are analytically equivalent to about 10^{-8} . *The wall geometry and its tangency*: 1.6×10^{-5} . *The start-line data*: agreed to 10^{-5} by two independent interpolation routes; refining the polyline resolution from 10^{-4} to 5×10^{-6} leaves the drift unchanged. *The strip above the initial-value line*: replacing the blend by a uniform state changes nothing, and the cone of dependence forbids it from reaching the wall anyway. *GENO's discretization*: a diagnostic GENO build with the fan step *and* the curve step both halved moved GENO's wall by 3.5×10^{-6} , so GENO is step-converged (the sources were then reverted and the binary restored, md5-verified). *Our own resolution*: halving each of our two step counts moves the deviation by less than 10% of itself.

Both fields satisfy the same trapezoidal march relations to 10^{-8} – 10^{-9} (an audit of the residuals run on *both* meshes). That proves the two discrete systems are algebraically equivalent — and it also proves that such an audit is *tautological* as an accuracy check: each code solves its own relations exactly, so a residual audit can never see a slowly accumulating bias. Worth stating plainly, because it looks like a strong check and is not. Finally, a control march with the axisymmetric source term switched off sizes the drift: it is 0.5% of what that term contributes to the wall velocity (which is itself -5.4% of q).

Attribution stays honestly *open*. The candidate mechanisms live precisely where refinement does not vary anything — on GENO's side the bookkeeping of re-interpolating during its backward march, on ours the interpolation along the chord found by the foot search — and from outside, at this level, they cannot be told apart. The owner-side question is filed with GENO. For this program the operational consequence is the one already adopted: cross-code comparisons with GENO carry the measured three-to-four-digit agreement level — far above the truncation error of either code, far below anything the optimizer's gradients are sensitive to — and the rejectors keep the instrument honest at that level.

Chapter 13

Free-vortex swirl

Everything so far assumed the gas moves in a plane containing the axis: forward and outward, never around. A rotating detonation engine (RDE) — a combustor in which a detonation wave runs continuously around an annular chamber — does not oblige: its exhaust comes out *spinning*. This chapter adds that spin to the solver in the one form that costs no new assumptions, shows by proof and by measurement that the whole certified machinery survives it, and states plainly what the extension does and does not model.

13.1 The question

Because the detonation wave sweeps around the chamber, it drags the gas with it: the exhaust carries an *azimuthal velocity* w — the component of velocity that goes *around* the axis, perpendicular to the plane through the axis in which the solver works (that plane is called the *meridional* plane, and q_m denotes the speed in it). The theory corpus guarantees (its statement N6-2) that for one particular swirl profile — the *free vortex* — the entire certified march structure survives untouched, and that anything beyond it is a class of flow this method cannot represent at all, and which belongs to the CFD-adjoint track.

A free vortex is the swirl of a bathtub drain far from its core: the product of radius and azimuthal velocity, $\Gamma = y w$, is *one and the same number everywhere in the flow*. That product is called the *circulation*; it measures how much the gas circles the axis, and physically it is the angular momentum per unit mass. So the swirl is fastest near the axis and dies away outward, and it is fixed by a single scalar Γ .

The question of this chapter: can the engine carry swirl, with the guarantee *measured* rather than assumed — and with the swirl turned off ($\Gamma \rightarrow 0$) recovering the certified swirl-free march exactly?

13.2 How it works, in full

13.2.1 Why free vortex is the one free extension

Three exact facts make the free vortex special.

First, the equation for momentum around the axis says that $y w$ is carried unchanged along each streamline. If it is one constant *everywhere*, that equation is satisfied identically and $w = \Gamma/y$ is known from position alone — no new unknown enters the march.

Second, the flow stays *irrotational*: fluid parcels are carried along and deformed, but they do not spin about their own centres. This is the property the whole solver rests on — it is what makes the flow describable by the two characteristic families of Chapter 3; a flow whose parcels do spin is called *rotational*, and for it the method of characteristics no longer applies. That is exactly what lies beyond the free vortex. Crocco's theorem, which ties vorticity to gradients

of total enthalpy and entropy, leaves only an azimuthal component possible when both are uniform, and the condition $V \times \omega = 0$ then forces even that to vanish. So the meridional flow stays irrotational, and the certified two-characteristic structure is not an approximation here but exact.

Third, working the gas-dynamic equations through with $w = \Gamma/y$ shows that the meridional problem changes in exactly two places and nowhere else:

- **the state:** the kinetic energy of the spin joins the energy budget, $h = h_0 - q_m^2/2 - \Gamma^2/(2y^2)$, so the thermodynamic state now depends on the radius y as well as on the meridional speed q_m ;
- **the source:** spinning gas must be held in by a pressure gradient pointing inward, and that centrifugal contribution adds to the term that axisymmetric flow already carries — the *axisymmetric source term*, the extra piece in the compatibility relations that accounts for the flow area opening out as y grows, absent in planar flow. It becomes $c^2 v/y \rightarrow (c^2 + \Gamma^2/y^2) v/y$.

The characteristic directions keep their form, now at the *meridional* Mach angle, and the compatibility coefficients in (u, v) are untouched.

13.2.2 The implementation is two changes, and only two

The state change costs nothing new. With the augmented speed $q_{\text{eff}} = \sqrt{q_m^2 + \Gamma^2/y^2}$ the energy closure reads $h = h_0 - q_{\text{eff}}^2/2$, exactly the swirl-free form, so the certified thermodynamic machinery applies verbatim to q_{eff} and only the Mach number is rebuilt from the meridional speed. The free jet edge — the boundary where the exhaust meets the atmosphere, on which the pressure is prescribed — keeps its certified parameterization: $p = p_a$ fixes the *total* speed, the same scalar as without swirl, so the edge's meridional speed follows an explicit law, $q_m(y) = \sqrt{q_{p_a}^2 - \Gamma^2/y^2}$.

The three cell types are mirrored with the swirl state and source and then run through the *unchanged* certified march driver through an additive seam, whose defaults reproduce the certified path exactly — both plug-march suites were re-run after the seam and still PASS.

One numerical lesson surfaced and is fixed in the swirl cells. The cell's small nonlinear system mixes rows of very different size: geometry rows of order one against compatibility rows of order u^2 . The damped Newton iteration — which accepts a step only if a norm of the residual decreases — can then stall one step short of the root, because the step that fixes the position trades a tiny increase in the compatibility rows against a geometry correction of order 10^{-7} , and the damping rejects it forever. Dividing the compatibility rows by a constant reference speed — a pure rescaling of the equations that moves no solution — restores certified convergence (worst cell 0.009 of tolerance).

13.2.3 What this is, and is not: the spiral phase pattern

The free vortex models the *time-mean* azimuthal velocity of the exhaust. It does not model the *spiraling of the phase pattern*: in a real RDE the surfaces of constant phase are helices — at one instant, different axial stations (and different azimuths) are at different points of the cycle, because the wave rotates while the flow transits. The cycle layer of Chapter 9 is deliberately *quasi-steady*: each phase fills the whole nozzle as a steady flow, and the cycle is an average over the phases with no clock. Two distinct physical effects are set aside by that closure. First, *unsteadiness within a meridional ray*: the phase seen at station x lags the injector phase by the transit time, and if the cycle period is comparable to the time gas spends in the nozzle, the flow never equilibrates to any phase — the quasi-steady model is the limit of vanishing reduced frequency, and for representative RDE numbers (kHz rotation against sub-millisecond residence) that frequency is of order one, so this is a real modeling limit, stated. Second,

azimuthal coupling: the helical arrangement tilts wave fronts out of the meridional plane — genuinely three-dimensional gasdynamics that no per-phase axisymmetric model can see. Both effects carry a partial mitigation and a clean escape route. The mitigation: if meridional rays were decoupled and quasi-steady, the *time average* at each station would equal the average over phases regardless of how the phases are arranged in space — the spiral reshuffles *when*, not *what*; the cycle-averaged objective is insensitive to the arrangement at that order. The escape route: both departures live exactly where the corpus already points — the unsteady three-dimensional CFD track, whose simulations contain the spiral natively and for which this engine is the oracle bank.

13.2.4 The worlds

Two test worlds, both on the real-gas tables, both under a strong swirl ($w/u = 0.35$ at the inner radius).

The *radial-equilibrium duct* is an exact known answer. In a straight annular duct the swirling solution has *uniform* axial speed and no radial velocity at all: differentiating the energy closure along y and subtracting the radial balance between pressure gradient and centrifugal force, $dp/dy = \rho\Gamma^2/y^3$, gives $du/dy = 0$ exactly — while the entire radial structure of pressure, temperature, sound speed and Mach number is carried by the swirl term (Figure 13.1a). The march must *preserve* that state, and any drift is pure error.

The *expansion world* drops the wall smoothly under the same stream, so the flow is genuinely two-dimensional ($v \neq 0$) and the centrifugal source actually acts. The duct alone cannot see that source, because there $v = 0$ multiplies it away.

13.3 What was built

`validation/a1_swirl_march.py`: the swirl state (augmented speed) and its coefficients, the three mirrored cell processes with the row scaling, swirl-aware flux and wall-push integrals, the two worlds, and the suite below. The certified driver gained the additive seam (`cells=, q_edge=`); its own oracle and axisymmetric suites re-ran 6/6 and 5/5 after the change.

13.4 Results of record: 9/9 PASS

W-1, the switch-off regression: at zero circulation the swirl cells reproduce the certified cells' march to 2.1×10^{-13} , against a band of 10^{-9} relative — the certified per-cell Newton tolerance, meaning every cell's nonlinear system was solved to a residual inside a derived band. The only arithmetic difference is the augmented-speed square root.

W-2, the exact duct: all cells Newton-certified (worst 0.009 of tolerance), and the equilibrium is preserved to machine zero — axial-speed drift 1.6×10^{-15} , $|v|/u$ 2.3×10^{-15} , wall pressure 4.9×10^{-15} , edge position exact — inside Richardson bands, the allowed errors obtained by re-running with the station spacing halved and letting the scheme measure its own truncation error (Chapter 4).

W-3, the expansion world: mass closes to 7.8 out of 4.3×10^4 (band 16), and the momentum theorem in the axial direction closes within band — unchanged by swirl, because the centrifugal force is radial and the momentum around the axis decouples from it; checked, not assumed.

W-4, the two live controls: switching the swirl off moves the wall speed $8.6\times$ beyond the pair's own band, and dropping *only* the centrifugal source while keeping Γ in the state moves it $12\times$ beyond — so the new term is genuinely wired in and genuinely material (Figure 13.1b). R-1, the rejector: cells fed a corrupted circulation ($\Gamma^2 \times 1.10$) but marched on true equilibrium data leave the preservation bands by twelve orders of magnitude — caught, as required.

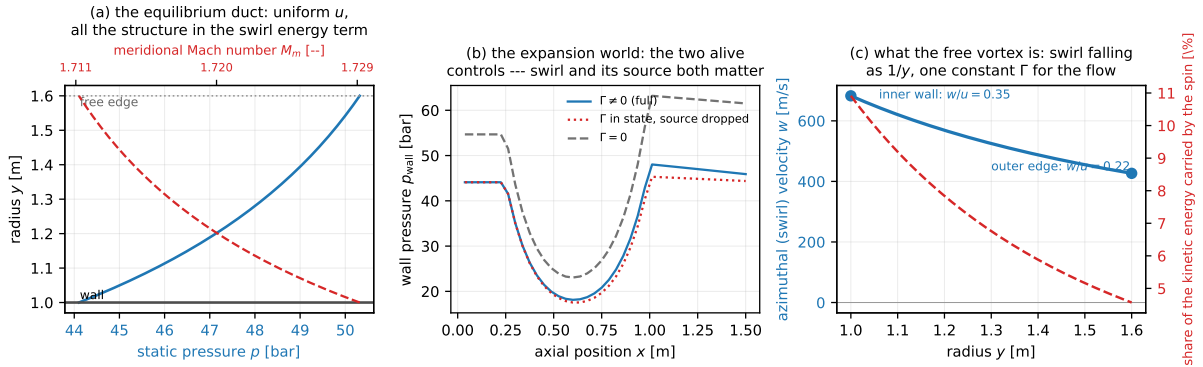


Figure 13.1: Free-vortex swirl, measured. (a) The radial-equilibrium duct, an exact known answer: radius y [m] vertically, static pressure p [bar] and meridional Mach number M [-] horizontally (two scales). Pressure rises and Mach number falls toward the outer radius even though the axial speed is uniform and the radial velocity is zero — the whole radial structure is carried by the swirl term in the energy budget. The march preserves this state to 10^{-15} . (b) The expansion world: axial coordinate x [m] against wall pressure p_w [bar], for three runs — full swirl, swirl with the centrifugal source deliberately dropped, and no swirl. The three curves separate far beyond the allowed error bands ($8.6\times$ and $12\times$), which is what proves both the swirl and its source term are wired in and material. (c) What the free vortex actually models: radius y [m] against azimuthal velocity $w = \Gamma/y$ [m/s] — the time-mean spin of the annular exhaust, fastest at the inner radius and decaying outward. It is a *mean*, not the spiraling phase pattern of the real engine, which is unsteady three-dimensional physics and belongs to the CFD track (Section 13.2.3).

Chapter 14

Comparing configurations, and the inlet angle

Everything up to here certified one machine at a time. This chapter asks the question a designer actually asks — bell or plug, how long, how much swirl, at what inlet angle — and finds that the hard part is not computing the answers but making them *comparable*. It also records the longest wrong turn in this program: four sessions spent chasing a defect that lived in a measuring instrument rather than in the flow.

14.1 The question, and why it is mostly a bookkeeping question

Thrust scales with mass flow. A bell computed in one posed world set against a plug computed in another compares the *worlds*, not the nozzle types. So before any comparison is meaningful a common basis has to be fixed and enforced:

- the same gas, the same chamber state (P_0, T_0) , the same ambient p_a ;
- the same **mass flow** — which carries a bonus, because for a choked throat $\dot{m} = p_c A_t / c^*$, so equal mass flow at equal chamber state means equal *throat area*, and therefore the thrust coefficient $C_F = F / (p_c A_t)$ and the specific impulse are directly comparable numbers with the size of the machine divided out;
- the same **envelope**: every configuration is allowed the same maximum radius. A plug given unlimited radius wins trivially on expansion alone.

Each configuration is then optimized over its *own* design variable: the bell over its area ratio, the plugs over where the spike is cut.

One subtlety makes the two sides the same physical quantity. For a rocket fed from a closed chamber the axial thrust equals the ambient-gauge momentum flux through *any* surface that spans the whole exhaust and encloses the hardware. For the bell that surface is the exit plane. For the plug the same theorem applies at the march start line: the flux there already contains the thrust of everything upstream, because the free jet boundary between the lip and that cut contributes exactly zero in the gauge ($p = p_a$ on it). So $F_{\text{plug}}(l) = F_{\text{in}}(x_0) + \int_{x_0}^l (p_w - p_a) 2\pi y (-dy)$ is the total thrust from the chamber through the truncation — the same object as the bell's.

14.2 Results of record

The bell wins by 0.282%, and that is resolved. The derived band on the thrust comes from refining the march itself: $J = 1.161610, 1.161481, 1.161454 \times 10^8$ N at three resolutions, successive differences 1.287×10^4 then 2.737×10^3 — a ratio of 4.70, confirming second order

configuration	F [MN]	C_F [-]	I_{sp} [s]	L [m]
bell, at its area-ratio optimum	116.474	1.4654	279.62	7.34
plug, full spike	116.148	1.4613	278.84	6.00
plug, truncated	116.148	1.4613	278.84	3.69
plug, truncated + swirl	111.839	1.4071	268.49	2.56

Table 14.1: Four configurations at one operating point on the common basis of Section 1: same gas, chamber, ambient, mass flow (hence throat area) and envelope radius. Thrust in meganewtons, thrust coefficient and specific impulse dimensionless and in seconds, length in metres.

— giving a Richardson band of 1.09×10^4 N, or 0.0094%. The bell–plug gap of 3.28×10^5 N is 30× that band.

The plug matches the bell in half the length, 3.69 m against 7.34 m, which is the aerospike result stated the way the thermodynamics permits it to be stated (Section 14.3).

Swirl costs 4.0%, an order of magnitude more than any geometric difference in the table — but the swirled spike runs down to 38× below ambient, a regime where a real flow would separate and this inviscid march cannot follow it. That number is indicative, and labelled so.

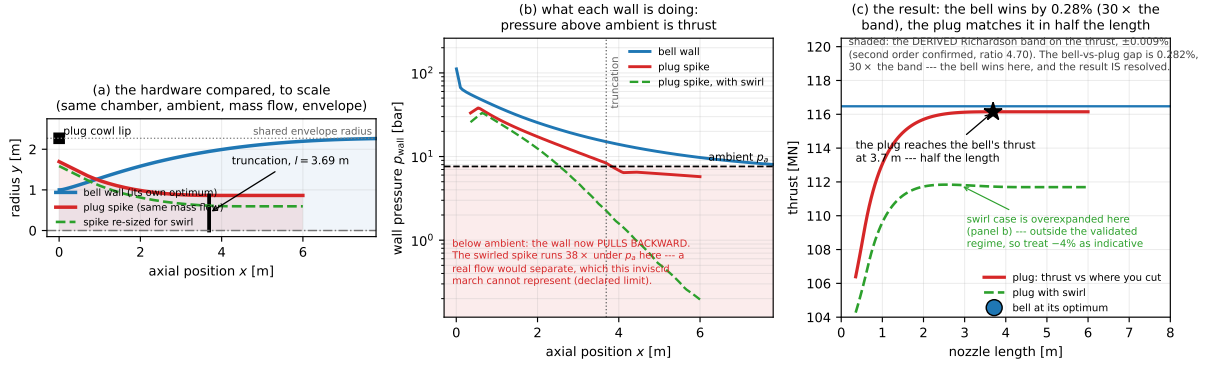


Figure 14.1: The configurations compared. (a) The hardware to one scale, axial position and radius in metres: the bell wall at its own optimum, the mass-matched plug spike, its truncation, and the spike re-sized for swirl. (b) Wall pressure in bar against the ambient line: above it the wall makes thrust, in the shaded region below it the wall pulls backward — and the swirled spike goes far into that region. (c) Thrust in meganewtons against nozzle length: the plug curve rises and saturates, reaching the bell’s thrust (blue) at about half its length. The shaded band is the *derived* Richardson band on the thrust, $\pm 0.009\%$; the bell–plug gap is 30× it.

14.3 Why they are so close: the thermodynamic ceiling

The near-equality of the coefficients is not a coincidence and not a disappointment. Both nozzles expand the same gas from the same chamber to the same exit pressure, and energy conservation alone then fixes the exit speed: $\frac{1}{2}q_e^2 = h_0 - h(p_a)$ gives $q_e = 2742.136$ m/s with no geometry in the statement. Since the basis forces the same mass flow, the ideal momentum thrust $\dot{m}q_e = 1.16474 \times 10^8$ N is exactly the bell’s measured thrust — an identity, because at its optimum the bell is perfectly expanded (the pressure term vanishes) and its exit is exactly axial (measured at $\theta_e = -10^{-16}$ in Chapter 5).

The bell sits *on* the ceiling. Geometry can therefore only lose against it, through divergence, non-uniformity and incomplete expansion — all effects of order one percent. That is the entire playing field of contour optimization, and it is why this program derives every tolerance instead of choosing one.

14.4 A certification gap the comparison exposed

Setting up the plug side required posing the flow the way a real aerospike does it: arriving at the lip aimed *inward* so the fan turns it back toward axial, with the spike descending toward the axis. Every certified axisymmetric plug world in this document poses the opposite — inflow axial, the fan turning it *outward*, the “spike” rising away from the axis (Figure 14.2a).

These are not mirror images. The axisymmetric source term is $\delta c^2 v / y$ and y is always positive, so $\theta \rightarrow -\theta$ is not a symmetry: the outward case runs with $v > 0$ and y growing, so the source weakens along the march, while the inward case runs with $v < 0$ and y shrinking, so it strengthens. The planar oracle is immune (no source, genuine mirror symmetry), so it is specifically the *axisymmetric* certification that only ever exercised one side.

Outward turning was never a candidate design — it was a test fixture. An outward-turning plug leaves its whole exhaust at $\theta_E > 0$ and collects only $\cos \theta_E$ of the momentum: 10.6% lost at the design turn of 26.7° , against nothing for an inward design. It is the natural way to write a closed-form corner-jet test field, and that is how it came to be the only case tested.

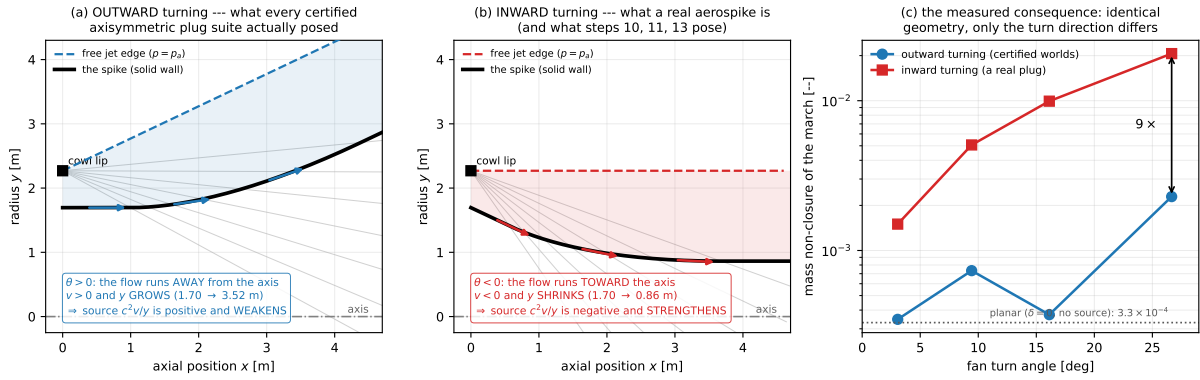


Figure 14.2: Outward versus inward turning, axial position and radius in metres. (a) What every certified axisymmetric suite posed: the flow arrives axial, the fan turns it outward, and the “spike” rises away from the axis; $v > 0$, y grows, the source weakens. (b) What a real aerospike is, and what this chapter poses: the flow arrives aimed at the axis, the fan turns it back, the spike descends; $v < 0$, y shrinks, the source strengthens. (c) The measured consequence, in mass non-closure of the march against fan turn angle: outward turning sits at the no-source floor (dotted) and stays flat, inward turning climbs to 2×10^{-2} .

14.5 The radial-flow oracle

To find out whether that difference was a defect, the program needed an exact axisymmetric solution with the flow running *toward* the axis. Spherical radial flow supplies one: every streamline is a ray through a point on the axis, the speed depends only on the distance from it, and continuity with the isentrope closes it as $\rho(q) q r^2 = \text{const}$. Putting the singular point upstream gives a diverging flow ($v > 0$); putting it downstream gives a converging one ($v < 0$). One formula, so the two cases differ in nothing but the sign the equations see. And since the streamlines are rays, a cone is an *exact* wall.

`validation/a1_source_flow_oracle.py`, **6/6 PASS**. The free edge is excluded by construction: only points below a rigorous dependence bound — a straight line from the start line’s top at the steepest characteristic slope present in the field, which nothing from above can outrun — enter the comparison. Against the closed form, the median errors are

	converging ($v < 0$)	diverging ($v > 0$)
wall pressure	7.9×10^{-4}	5.2×10^{-4}
interior $ \Delta q /q$	3.4×10^{-4}	1.8×10^{-4}

The interior and prescribed-wall cells are clean for inward-turning flow. A useful trap surfaced in building the negative control: in radial flow *every* cone is an exact streamline, so perturbing the cone angle merely produces a different exact solution and can never reject. The valid rejector curves the wall off the ray, breaking the ray property, and separates by $29\times$. Even rejectors must be chosen by physics — the same lesson as Appendix A.2, met again.

14.6 The instrument, not the flow

The mass non-closure that started this investigation is largely a property of the meter. The conservation check integrated over the march’s *last column*, which is a characteristic, not a plane, and whose segment lengths are wildly uneven: on a descending spike the longest is 0.84 m against a mean of 0.064 m. Trapezoidal error scales as the square of the segment, so that one segment dominated. Three measurements settle it: the free edge’s own streamline flux is 4×10^{-11} — machine zero, so it leaks nothing; the wall leaks 0.016%; and the cells reproduce the exact solution above. Per-column inspection finds the cause: only the *first* marched column has a coverage hole, $26\times$ the mean row spacing when the wall rises and $39\times$ when it descends, against $0.5\text{--}1.0\times$ for every other column — because a column is a characteristic while a vertical start line is not, so the first column alone must bridge the two geometries.

The lesson is the one worth carrying: **when the quantity of interest has a derived band available directly, derive it directly.** The thrust’s uncertainty was inferred from a conservation meter measuring something else; refining the march and letting it measure its own error — the program’s own first rule — settled it in one run.

14.7 The inlet angle as a design variable

By owner directive the initial turning angle is no longer posed: it is part of the optimization. The RDE delivers its flow across a *straight* annular inlet between the spike start radius and the cowl lip, and the inclination of that inlet is the design variable.

The lip fan turn $\Delta\nu$ is fixed by the expansion (from M_i down to p_a) and does not depend on the inlet angle, so the exhaust angle is simply $\theta_E = \theta_i + \Delta\nu$; the sweep is reported in θ_E because that is what the physics cares about, and the classic design condition is $\theta_E = 0$. The basis of Section 1 is unchanged, with one addition: the axial mass flux through an inclined inlet scales with $\cos\theta_i$, so the annulus is re-sized at every candidate to hold the reference mass flow, and thrust is compared at a *fixed length* so that length is not silently traded against angle.

`validation/a1_inlet_angle_opt.py`. The blind sweep lands at $\theta_E = 0$ — the divergence prediction, recovered rather than imposed — and the measured penalty follows the $\cos\theta_E$ model in sign and order across the whole admissible window:

θ_E [deg]	measured	$\cos\theta_E$ model
−6	−0.365%	−0.548%
−4	−0.153%	−0.244%
−2	−0.031%	−0.061%
0	best	—
+2	−0.068%	−0.061%
+4	−0.224%	−0.244%
+8	−0.799%	−0.973%
+12	−1.753%	−2.185%

Two things are worth reading off. The measured penalty is consistently *smaller* than pure divergence, because the wall pressure distribution partially compensates — the trade the closed-form model does not contain. And the curve is asymmetric: turning inward past the optimum costs less than turning outward by the same angle, which is the quantitative form of the directive’s premise that outward-turning aerospikes are not competitive. The window is bounded on the inward side by geometry, not preference: at $\theta_E = -8^\circ$ the spike descends to $y = 0.32$ m and the march loses certification as the flow runs into the axis, where the axisymmetric source diverges.

Chapter 15

Status, what remains, and reproduction

This chapter is the ledger. It lists every test suite the program owns and its verdict, states exactly which quantities a design run may fix and which it may move, says what is not yet built, and gives the command that re-runs each result from scratch. Nothing here is new physics: it is the part a reader needs in order to check the rest.

15.1 Status of every suite

Step	Carrier (validation/)	Verdict	Chapter
Brick 1 march	a1_ideal_march_jax.py	certified	(prereq.)
Thrust functional	a1_thrust_functional.py	8/8 PASS	5
Corner instrument	a1_corner_instrument.py	5/5 PASS	5
ε -driver vs closed form	a1_driver_eps.py	4/4 PASS	6
Prescribed-wall march	a1_wall_march.py	5/5 + 2/2 PASS	7
GENO cross-code (O3.4)	g0_geno_crosscode.py	PASS	8
Rao multipliers vs AD (O3.3)	a1_o33_toc.py	9/9 PASS	8
Cycle layer (collapse + mock RDE)	a1_cycle_layer.py	5/5 + 6/6 PASS	9
Cycle corner meter (votes)	a1_cycle_corner.py	8/8 PASS	9
Free-jet cell	a1_freejet_unit.py	6/6 PASS	10
Plug march (oracle + axi)	a1_plug_march.py	6/6 + 5/5 PASS	10
Truncated plug under cycle	a1_plug_cycle.py	5/5 + 9/9 PASS	11
GENO full-field plug twin	a1_geno_plug_twin.py	8/8 PASS	12
Free-vortex swirl march	a1_swirl_march.py	9/9 PASS	13
Radial-flow oracle ($v < 0$)	a1_source_flow_oracle.py	6/6 PASS	14
Configuration comparison	a1_config_compare.py	see §14	14
Inlet angle as a variable	a1_inlet_angle_opt.py	6/6 PASS	14

Table 15.1: Every certified suite of Brick 2, its carrier script and its verdict of record. “ m/n PASS” counts the suite’s checks, rejectors included.

15.2 Design inputs and constraints, as of today

15.2.1 What a design run can fix

Designing means holding some quantities fixed and letting the optimizer move the rest. Every item below can be held fixed independently for a run; whatever is not held is what the optimization is allowed to change.

Gas and chamber: the property tables (any gas model inside their range of validity), the chamber conditions (P_0, T_0) — the pressure and temperature the gas would have at rest — and, for RDE runs, the whole cycle $P_0(\xi), T_0(\xi)$ with the weight given to each phase ξ of the cycle.

Environment: the ambient pressure p_a the nozzle exhausts into, vacuum included. For a plug nozzle it is not merely bookkeeping: it enters the flow itself, since the free jet boundary is the surface on which $p = p_a$.

Throat geometry: the throat radius y_t and the two arc radii r_{tu}, r_{td} that round the throat, all differentiable knobs. Holding y_t is the mass-flow constraint: a choked throat passes a fixed mass flow $\dot{m}$, so fixing its radius fixes the engine's flow rate.

Overall size: the area ratio ε — the exit area divided by the throat area, the single number saying how much the nozzle expands the gas (the one-knob driver of Chapters 6 and 9).

The wall: an output when the engine generates a contour; a fully given input in the inverse bell and plug modes; and a parameterized unknown when it is being optimized — today small bumps whose ends are pinned, spline control points next. Pinning the endpoint is the fixed-length constraint: it is exactly the problem Rao posed, and exactly what the stationarity test of Section 8.2 held fixed.

Modes: planar or axisymmetric; a single steady operating point or a cycle average. *Resolution:* the number of fan rays, stations and start rows — not design variables, but fixed per run and part of every derived error band.

15.2.2 Which constraints can be held simultaneously

On the JAX side, constraints are today imposed *by construction* rather than by a general constrained optimizer: mass flow by holding the throat, length by pinning the lip, ambient as posed data — and the optimizer moves whatever is left (today ε , or the wall through its bump basis). Holding several nonlinear constraints at once against the AD gradient needs the general machinery: a trust-region SQP driver — a standard constrained optimizer that takes a limited-size step each iteration and enforces the constraints to first order — which is on the remaining-work list, and for which the certified prescribed-wall paths were built.

On the GENO side, the RaoPlug module (its case type 8) accepts, since the two-constraint session, exactly *two* simultaneous constraints. The first is the *mass constraint*, always on: the design curve is terminated where the mass accumulated inside it equals the throat's target $p_c \pi y_t^2 / c^*$, and that condition is consumed by the point D at which the spike is cut. The second is *one* user-chosen design constraint, consumed by a bisection on the exit angle θ_E — area ratio, plug length, exit angle, thrust, exit Mach number, ambient pressure at the lip, or the base pressure p_b (matched against the value the corner condition implies at the mass-set cut, the meaningful pairing with the mass constraint).

Two is not an implementation limit but a *theoretical* one. Rao's spike family has exactly two degrees of freedom at a fixed lip radius — the exit angle θ_E and the cut point D — so two conditions exhaust it, and a third would be over-determined. The one honest route to a third simultaneous constraint is to free the lip radius y_E as a scale degree of freedom, an outer bisection already sketched in GENO's design notes and not yet built.

15.3 What remains, in order

The GENO thread is now closed end to end. The owner's decision was to solve the two-constraint problem, and the GENO session landed it: the design curve terminates at the prescribed throat mass, the cut point supplies the second degree of freedom with the corner condition returning the base pressure it implies, four rigor defects were fixed on the way, and three independent checks PASS (the duality between the two ways of writing the endpoint condition, 2×10^{-5} ; the exact planar limit, bit for bit; an independent Python integration of the design curve, three to four

digits). One item did not close: the numbers tabulated in the 1961 spike paper as the registry quotes them proved unverifiable and internally inconsistent (GENO's N-74), so the owner must procure the original paper. The full-field comparison — our plug march run on GENO's own spike — is now certified 8/8 in Chapter 12, with the measured three-to-four-digit difference between the two codes characterized and its cause filed as an open question on GENO's side (Section 12.5).

The free-vortex swirl extension is certified at march level in Chapter 13: the structural guarantee measured rather than assumed, an exact duct solution reproduced, and the switch-off regression at 10^{-13} . Using it inside the plug and cycle studies is now a posing choice, not new machinery. Beyond the free vortex the flow is rotational and belongs to the CFD-adjoint track.

1. **Measured-cycle input:** a data swap — mind the table-range rejector of Chapter 9.
2. **Spline control-point driver** (trust-region SQP) on the verified prescribed-wall paths.

15.4 Reproduction

Each carrier script is self-contained and prints its own verdict; runs are cached where noted, so re-runs are cheap. On host s2:

```
cd RDE_Design-rde-nozzle-program
.venv-a1/bin/python validation/a1_ideal_march_jax.py      # Brick 1
.venv-a1/bin/python validation/a1_thrust_functional.py    # the score and the
compass
.venv-a1/bin/python validation/a1_corner_instrument.py    # the score and the
compass
.venv-a1/bin/python validation/a1_driver_eps.py           # the first optimization
.venv-a1/bin/python validation/a1_wall_march.py           # the wall becomes an
input
.venv-a1/bin/python validation/g0_geno_crosscode.py       # the referee
.venv-a1/bin/python validation/a1_o33_toc.py              # the referee
.venv-a1/bin/python validation/a1_cycle_layer.py          # the cycle layer
.venv-a1/bin/python validation/a1_cycle_corner.py         # the cycle layer
.venv-a1/bin/python validation/a1_freejet_unit.py         # the plug march (cell)
.venv-a1/bin/python validation/a1_plug_march.py           # the plug march
.venv-a1/bin/python validation/a1_plug_cycle.py           # truncated plug + cycle
.venv-a1/bin/python validation/a1_geno_plug_twin.py       # the GENO plug twin
.venv-a1/bin/python validation/a1_swirl_march.py          # free-vortex swirl
```

This document itself is built from docs/brick2_doc_src/: make_figures.py regenerates the data figures from the cached run data, and build.sh compiles main.tex (Tectonic engine) and installs the PDF as docs/rde_nozzle_A1_brick2_thrust.pdf.

Appendix A

The problems met, and how they were fixed

A document of record that showed only the working version would be hiding its best evidence. Every test in this program is built to be able to say *no*: its pass band is derived from the scheme's own measured error rather than chosen by hand, and each suite plants deliberate corruptions — *rejectors* — that must make it fail. A bug the suite caught is therefore not an embarrassment but a measurement: proof that the instrument has teeth, and a record of what the physics does when it is got wrong. The main chapters show each step as it now works; this appendix collects, step by step, what went wrong on the way, how it was diagnosed, and what was changed. Nothing here is anecdote — every item was caught by a test and closed by a measured fix.

A.1 Step 1: the unclosed control surface

Thrust is computed as a balance over a *control surface*: an imaginary closed surface drawn around the flow, over which the momentum carried through and the pressure pushing on it are added up. One test of that bookkeeping is the *gauge test* — add the same constant to the pressure everywhere and nothing may change, because a constant integrates to zero over a surface that is genuinely closed. It failed on its very first run (Chapter 5).

Cause: the wall integral of route 2 started at the first wall *point* of the mesh, leaving a microscopic sliver of wall between the throat lip and that point uncovered. The surface was not closed, so a constant no longer integrated to zero (the red circle in Figure 5.1). Fix: close the surface at the throat lip. The momentum residual R_{mom} — the disagreement between the two independent thrust routes — got *tighter* once the sliver stopped hiding inside it: the bug had been absorbing part of the mechanical balance.

A.2 Step 6: the rejector corrected by physics

A rejector is only useful if the corrupted case it poses is one the test could plausibly fail. The first negative control for the first-integral test of Section 8.2 — the check that a particular combination f_2 of speed and angle is constant along the last characteristic of an optimal nozzle — used an arbitrary vertical cut through the flowfield, expecting f_2 to vary along it. It barely did: much of the smooth tail of a nozzle is nearly f_2 -flat, so a vertical cut is a *weak* negative control. The sharp one is the *wall*, a streamline along which f_2 climbs by 64%; the rejector was rebuilt on it. Lesson: even rejectors must be chosen by physics, not by convenience.

A.3 Step 4: the table-kink ripple

The gradient stage of Chapter 7 first posed the obvious test: compare the derivative produced by automatic differentiation (AD) with a finite difference — the objective recomputed after nudging one input — at shrinking nudge sizes, expecting agreement to improve. It failed in a way no step size could fix. The finite difference diverged as the step fell below the width of one cell of the gas-property tables, while the AD value oscillated.

Diagnosis: the tables are interpolated piecewise-linearly, so the function the code actually implements has small kinks at every table cell boundary, and the *exact* derivative of that function carries a genuine ripple at the table-cell scale — measured at a standard deviation of 44, i.e. 0.010% of the gradient and 1.2×10^{-8} of the thrust J (Figure 7.2). Neither number was wrong; the *test* was. A pointwise AD-versus-finite-difference test on tabulated thermodynamics is ill-posed.

The honest certificate separates the smooth curvature of J (physics) from the ripple (interpolation) by fitting a straight line over a sweep of nudge amplitudes and testing the resulting envelope identity instead — the certificate the suite now carries. A first analysis also mislabeled a smooth 2.25% curvature as “ripple”; the linear-fit separation corrected that to the true 0.010%. Cubic table interpolation would remove the ripple at the source. The same trap awaits any differentiable solver built on tabulated properties, the CFD-adjoint track included.

A.4 Step 7: three cycle-layer traps

The cycle layer (Chapter 9) — which represents an RDE by a family of steady chamber states, one per phase ξ of the wave’s passage — met three traps, each now guarded by a rejector.

1. **The enthalpy zero-point.** Enthalpy has no absolute zero: every table fixes its own reference state, and only differences are physical. Building each phase’s chamber state *absolutely*, as $h_0 = h(T_0)$ read straight off the raw table, mis-stated the state by 19%. Fix: build phase states *relative* to the tables’ own reference ($h_0 = h_0^{\text{tab}} + h(T_0) - h(T_s^{\text{tab}})$, and likewise for entropy with its $R \ln(P_0/p_s)$ term). Guard: doubling the chamber pressure must exactly double every pressure in the field, $p(2P_0)/p(P_0) = 2$, now verified to 1.6×10^{-15} .
2. **The pressure-blind peak design.** The first attempt to “design at the peak phase” held the exit Mach number fixed — which is not designing at the peak at all, because it ignores where that phase’s own exit pressure would match the ambient. Fix: the peak design solves $p_e(\varepsilon) = p_a$ for the peak phase’s own state (`eps_star_at_pa`).
3. **The silent table clamp.** A first mock-RDE run posed a peak chamber temperature $T_0 = 4162$ K — past the 3900 K edge of the tables — and the interpolation routine `np.interp` *clamps silently* at the edge instead of complaining. The run completed, with quietly flattened thermodynamics. Fix: the mock cycle was re-posed inside table validity (peak 3850 K), and a table-range rejector now fails any phase whose T_0 leaves the tables — the first thing to check when a measured cycle is fed in.

A.5 Step 7 (assembly): the free-jet sign trap

On the free jet edge — the boundary where the exhaust meets the atmosphere, on which the pressure is prescribed and the position is solved — the first version of the cell wrote the speed condition by substituting moduli, $v = +\sqrt{q_{p_a}^2 - u^2}$ (Figure 10.1b). That form silently allows only outward turns, because the square root is never negative; a *compressing* jet must turn inward. The unit-test rejector U-3 — a compressing edge posed on exact data — caught the

clamped spurious root. The cell was reformulated with the turning *angle* θ_4 as the unknown, so the sign of the turn lives inside the unknown itself and both directions are reachable — the formulation certified 6/6 in Chapter 10.

A.6 The plug assembly: four structural lessons

The plug march of Chapter 10 reached its certified form through four measured structural failures. Each was caught by the closed-form oracle — an exactly known answer the march must reproduce — diagnosed to a mechanism, and closed by a design change. The design presented in Chapter 10 *is* the sum of these fixes.

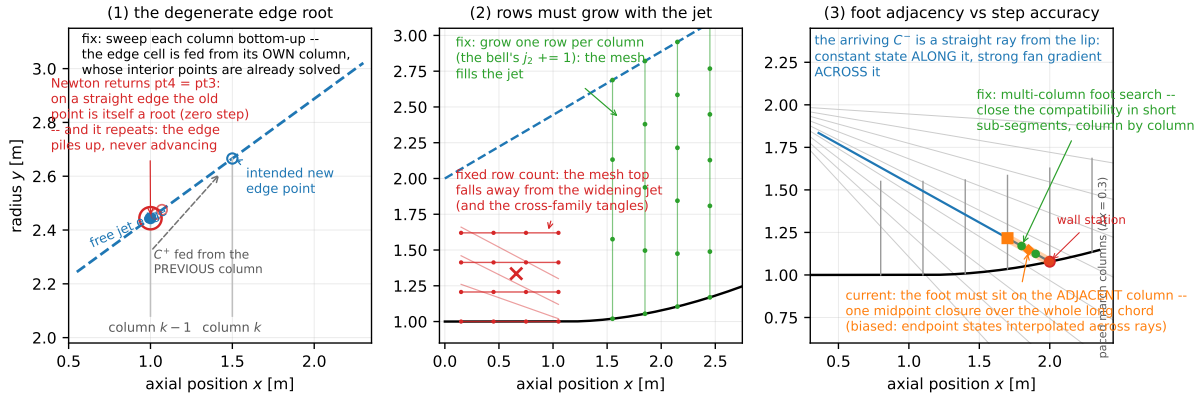


Figure A.1: The three measured lessons, drawn in the exactly known test world; all panels are axial coordinate x [m] horizontally against radial coordinate y [m] vertically, with the mesh the march actually built. (1) The degenerate edge root: successive edge points pile up at one place instead of advancing (Section A.6.1). (2) With a fixed number of mesh rows per column, the top of the mesh falls away from the widening jet (red); adding one row per column — the same device the bell march uses — keeps the mesh filling the jet (green). (3) The characteristic arriving at the wall is a straight ray from the lip: the state is constant along it and varies strongly across it. With the foot of that ray forced onto the neighbouring column, the compatibility relation was closed by a single midpoint rule over a long chord whose end states are interpolated *across* rays — systematically biased. The fix is the multi-column foot search of Chapter 10.

A.6.1 Lesson 1: the degenerate edge root

In the first assembly the free-edge cell was fed from the *previous* column, and on straight stretches of the edge the Newton solve returned the new point exactly on top of the old one (pt4 = pt3) — a zero step, repeated at every column: the edge piled up and never advanced.

The mechanism (Figure A.2) is worth stating because it is generic. Newton does not march, it *solves*: anything that zeroes the residuals is a valid root, including a useless one. At pt4 = pt3 the streamline condition is satisfied identically ($0 = 0$), the pressure condition already holds (pt3 is on the free boundary by construction), and the third equation turns on what pt3 *is* in the mesh. It was created, one step earlier, as the exit of a characteristic segment launched from the previous column's top data — the mesh literally records a characteristic terminating at pt3. A sender taken from that same data therefore lies on pt3's own characteristic (exactly so where flow and edge are straight, since there the discrete segments lie exactly on the straight characteristics; to second order where they curve — which is why the bug hid until the edge ran straight), and re-drawing that line can only re-find its recorded exit.

The fix: sweep each column from the bottom up and feed the edge from its *own* column's top interior point — a point on the *next* characteristic of the family, whose line is not asked to

hit pt3 at all but to cross the *streamline* through pt3. That is a shallower line, which a steeper line from below catches strictly downstream.

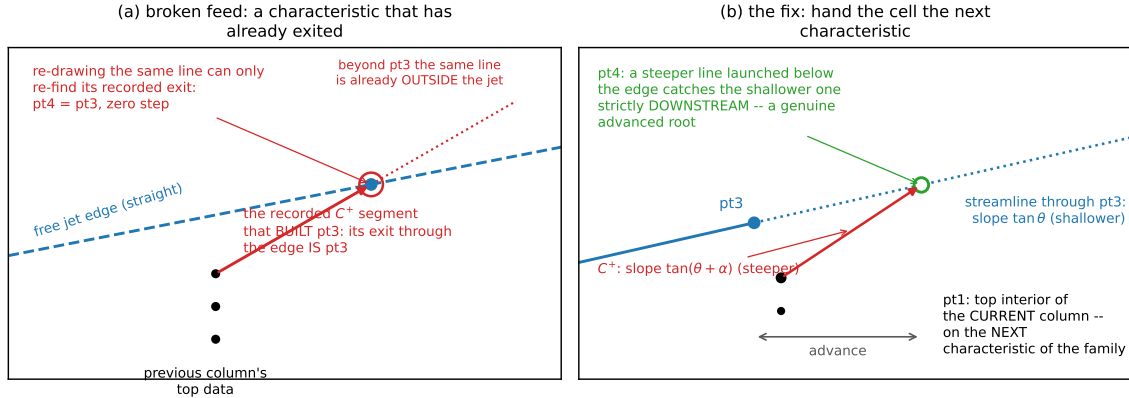


Figure A.2: The geometry of the degenerate edge root; both panels are axial coordinate x [m] against radial coordinate y [m], drawn at the scale of two mesh cells. (a) The broken feed: pt3 exists in the mesh only as the recorded exit of a characteristic segment from the previous column's top data, so a sender taken from there sits on pt3's own characteristic — re-drawing that line can only re-find its recorded exit, giving $pt4 = pt3$ and a zero step. (b) The fix: the current column's top interior point sits on the *next* characteristic, and its line must cross the *streamline* through pt3 — a shallower target, caught strictly downstream, so the root is genuinely advanced. What to compare: the angle between the two lines in each panel.

A.6.2 Lesson 2: rows must grow with the jet

The first mesh carried a fixed number of rows per column. But the jet widens downstream: the nearly horizontal characteristics of one family dragged the fixed rows down while the free edge climbed away from them, until the top of the mesh fell out of the jet altogether and the two families tangled (Figure A.1, panel 2). The bell march survives the same geometry by adding one row per column; the plug march now does the mirrored same — each column's free-edge point becomes a new top row, so every column brings one new characteristic and the mesh keeps filling the jet.

A.6.3 Lesson 3: foot adjacency vs step accuracy

With the edge and the rows fixed, the wall cell still failed its pressure check downstream, and a probe *named* the defect: wall angles exact to four decimals everywhere, but wall speeds biased and drifting once the segments grew long.

The first scheme confined the wall cell's *foot* — the point on the previous column from which the arriving characteristic departed — to the first segment of the adjacent column. To keep the foot there the columns had to be paced, with the axial spacing allowed to grow up to a clamp of 0.30. A single midpoint closure over a chord that long, crossing the expansion fan, is systematically biased: the chord itself lies along a straight ray from the lip, along which the state is constant, but its end states are interpolated *across* rays on the column, and the midpoint rule compounds that transverse error over the whole long segment. Intermediate rounds (aligned columns, row growth, pacing) raised the wall-pressure agreement from 22% to 55% of stations and made the failure reproducible at the same station at both resolutions — the signature of a systematic closure defect rather than noise.

The fix is the multi-column foot search now in the march (Chapter 10): columns are placed at the fine prescribed stations, and the wall cell searches the whole previous column, stepping

further back if needed, for the segment that brackets its arriving characteristic. Chords shrink from the 0.30 clamp to the station spacing, and a midpoint closure's error falls with the square of the chord. That single change took the wall-pressure agreement from 55% to 98% of stations and brought the edge angle into band.

A.6.4 Lesson 4: the ghost rows, and why one oracle is not enough

With the foot search in place the suite still read 3/5: every cell Newton-certified, edge and wall pressure in band — and mass conservation off by +9%. The probe found a whole limb of mesh *below the spike*, carrying the upstream uniform state (Figure A.3).

Mechanism: with fine stations, a near-wall row's characteristic terminates on the wall *between* two stations. The march kept prolonging that row anyway, and for such a row the only crossing Newton can still find is the *backward* one — a perfectly valid root of the cell's equations, converged and certified, on the wrong branch. (The foot search was already detecting the terminations — its foot row climbed one row per column — but nothing acted on them.) The fix is the row *consumption* of Chapter 10: at a solid wall the incoming family terminates, so rows of the previous column below the new foot are retired and the column re-indexed — the exact dual of the edge adding one row per column. Mass then closes to 0.026%.

The methodological lesson is the reason this appendix exists. The defect was invisible to the pointwise oracles (P-1, P-2) and to Newton certification, because every individual cell really had been solved correctly; only the *global* mass balance saw it. Suites need conservation checks precisely because wrong-branch roots certify locally.

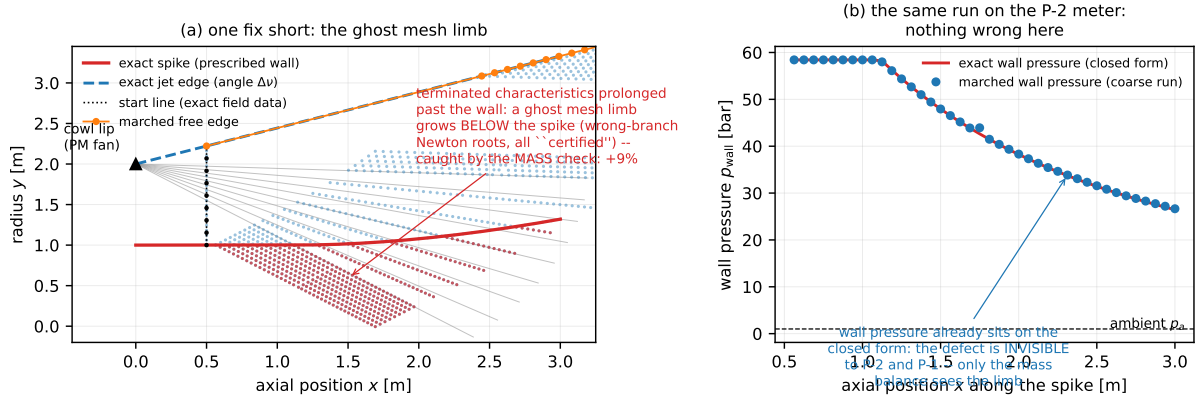


Figure A.3: The ghost-limb regression, reproduced live with the carrier's diagnostic switch (`consume=False`). (a) Axial coordinate x [m] against radial coordinate y [m]: characteristics that should have terminated at the wall are prolonged past it and grow a limb of mesh below the spike (red points) — every cell of it Newton-certified, all of it on the wrong branch. (b) The same run measured on the wall: x [m] against wall pressure [bar], against the closed-form answer. The two lie on top of each other — which is the point: the defect is invisible to the pointwise checks and was caught only by the mass balance, off by +9%. Compare with the certified state, Figure 10.3.

A.7 Step 10: the posing lessons of the plug cycle

The truncated-plug study (Chapter 11) reached its certified form through three measured corrections — two about how the problem must be *posed*, and one where the theory itself rejected a wrongly transplanted test. All three are drawn in Figure A.4.

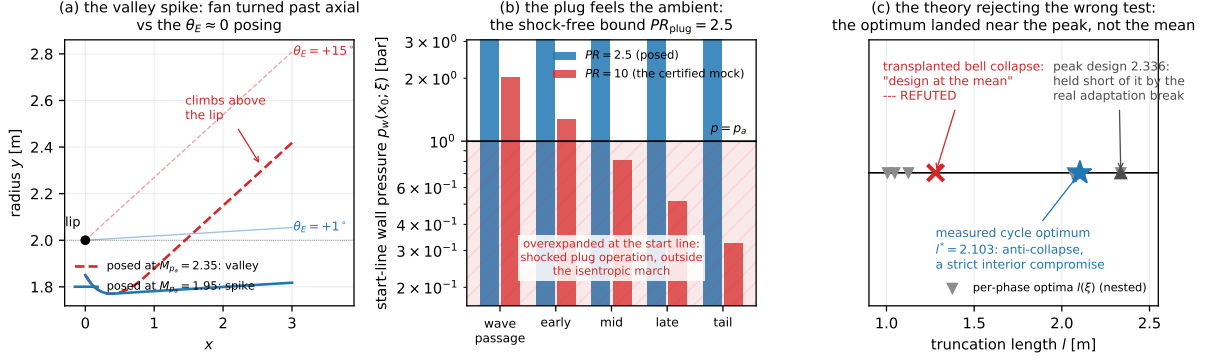


Figure A.4: The three posing lessons of step 10, drawn from the engine’s own machinery and the numbers of record. (a) Axial coordinate x [m] against radial coordinate y [m]: the valley spike. Posing the same lip fan at $M_{pa} = 2.35$ turns the flow past axial ($\theta_E = +15^\circ$) and the “spike” streamline climbs *above* the lip instead of descending; the corrected posing ($M_{pa} = 1.95$, $\theta_E \approx 0$) gives a genuinely descending spike. (b) Cycle phase (the five phases, from wave passage to tail) against the wall pressure each phase produces at the start line [bar], with the shared ambient p_a drawn as a line. At a chamber pressure ratio of 2.5 every phase sits *above* the ambient — underexpanded, and free to adapt inside the domain. Rescaling the same ladder to the certified $PR = 10$ mock cycle puts the low phases far *below* p_a : a shocked start, which the shock-free march cannot represent. That gap is what bounds the usable cycle. (c) The truncation length l [m] as a number line, marking where each candidate answer falls: the expectation transplanted from the bell (design at the mean phase, red cross) is refuted by the measured cycle optimum $l^* = 2.103$ — an interior compromise sitting near, but strictly short of, the peak design — with the per-phase optima (∇) spread out in the order the plug theorem predicts.

A.7.1 The valley spike

The first world posed the ambient pressure too low (the mean-phase pressure at Mach 2.35). The fan at the cowl lip then turned the flow by $+40^\circ$ — past axial — and the streamline meant to be the spike dived, reversed and climbed *above the lip*: a valley, not a spike, with the wall approaching the free edge and every downstream check breaking on the geometry. The fix is the classic plug design condition: pose the ambient so that the fan turn roughly cancels the incoming flow angle, i.e. $\theta_E \approx 0$ (the flow leaves axially, adapted to the ambient), which makes the streamline genuinely descending.

A.7.2 The plug feels the ambient: the shock-free cycle bound

A bell nozzle never feels the ambient: its flow is computed to a fixed exit whatever the outside pressure, and p_a enters only the score. A plug feels it directly, through the free edge where $p = p_a$ is imposed on the flow itself.

Running the certified mock cycle, whose chamber pressure ratio between the strongest and weakest phase is $PR = 10$, against one shared ambient put the low phases $6\times$ overexpanded already *at the start line* — a regime with shocks, outside what an isentropic march can represent. The failure was measured, not hypothetical: the free-edge cells blew up and the garbage descended row by row to the wall. Newton certification caught it, and the scaling-law check read a spread of 14.6 where it should have read machine zero.

A cycle for a plug study must therefore be posed *shock-free*: every phase underexpanded at the start and adapting within the domain. With one shared ambient that bounds the usable pressure ratio — here $PR = 2.5$. The theory’s statements do not depend on PR , and the corpus already conditions on shock-free flow; but the bound is a real modeling limit worth stating,

because cycles as violent as $PR = 10$ put their low phases into shocked plug operation that this engine cannot yet represent.

A.7.3 The theory rejecting the wrong test, twice

Two of the first draft’s checks failed — and both failures were the theory working, not the code.

First, the draft transplanted the bell’s collapse result (“the cycle optimum is the design for the mean-pressure phase”) to the plug. It failed: the measured optimum sits at $l^* = 2.10$ against a mean-phase design of 1.28. That is the *anti-collapse* — the opposite side of the theory’s dichotomy — because the collapse result is proved only for a fixed, fully flowing wall, which a truncated plug with a free edge is not. The check was inverted into the anti-collapse oracle now in Chapter 11.

Second, the draft required the per-phase votes at l^* — each phase’s wall pressure minus the ambient at the truncation plane — to vary monotonically with phase and change sign once, as certified for the bell in Chapter 9. They do not: on a plug the adaptation waves make the wall pressure oscillate about p_a , so pointwise votes sample an oscillation and the wave-passage phase can vote negative at a length below its own optimum. The theory claims only the averaged balance and the compromise structure, and exactly those pass; the over-specified checks were replaced by theorem-faithful ones (a genuine spread of the per-phase optima; l^* a strict interior compromise).

The lesson generalizes: an oracle may be transplanted from one problem class to another only together with the hypotheses that prove it.

Bibliography

- [1] G. V. R. Rao, “Exhaust Nozzle Contour for Optimum Thrust,” *Jet Propulsion*, Vol. 28, No. 6, 1958. (The corner condition, the multipliers λ_2, λ_3 , and the terminal characteristic as control surface.)
- [2] G. V. R. Rao, “Spike Nozzle Contour for Optimum Thrust,” *Planetary and Space Science*, Vol. 4, 1961.
- [3] M. J. Zucrow and J. D. Hoffman, *Gas Dynamics, Volume 2: Multidimensional Flow*, Wiley, 1977. (Unit processes; the inverse-wall point.)
- [4] *GENO: A Method-of-Characteristics Design Code for Supersonic Nozzles — Theory and Implementation*, DIMA, 2026. ([GENO/docs/GENO_theory_and_implementation.pdf](#) — the independent referee of Chapter 5, and the format model of this document.)
- [5] *RDE Nozzle Program: Theory*, this repository, [RDE_Design_NozzleProgram_Theory.pdf](#). (The collapse dichotomy, the averaged corner condition, the swirl identities, and the conditional registry — corpus tags T3/T4, T7, N6, C-O33.)